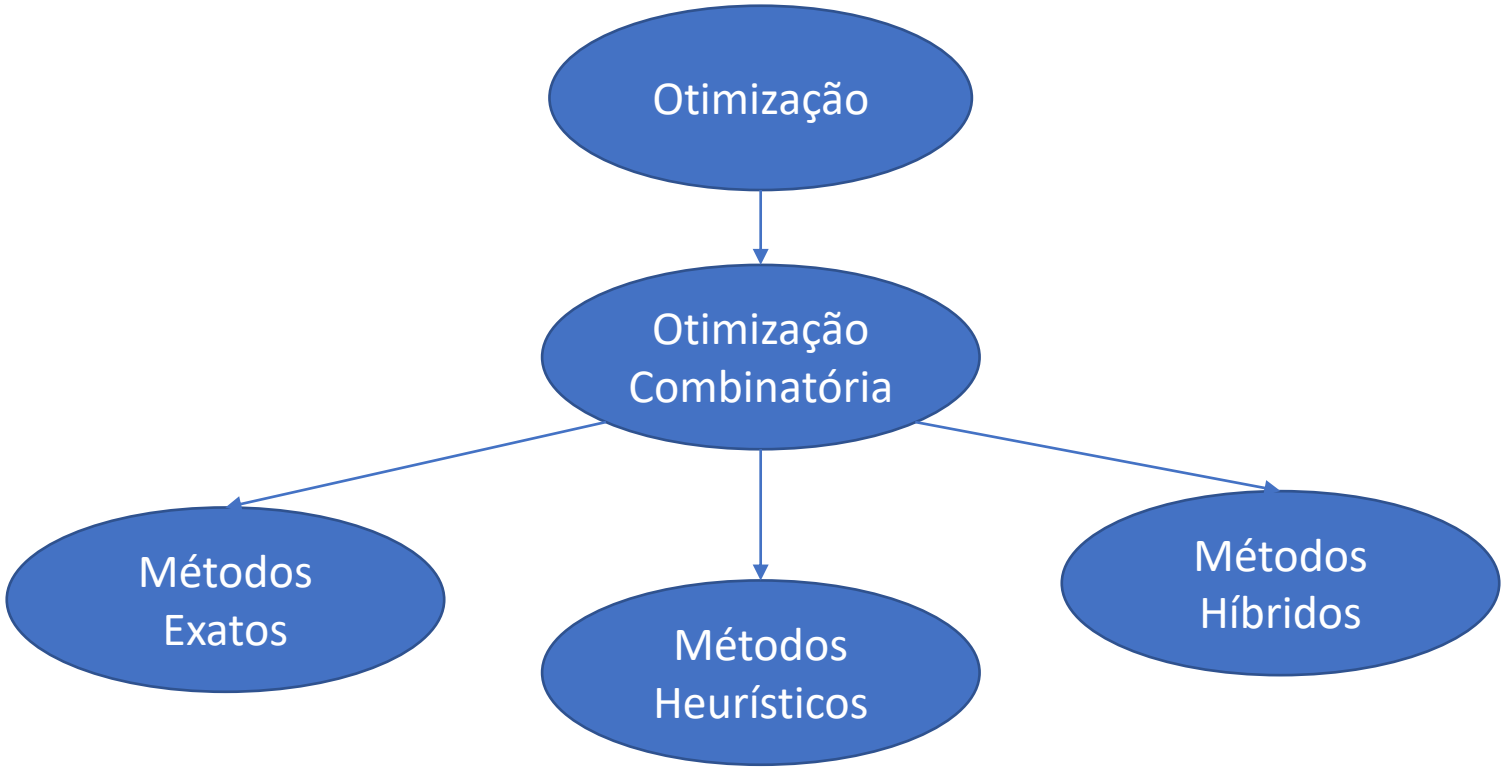
Engenharia de Sistemas e Computação PESC/Coppe

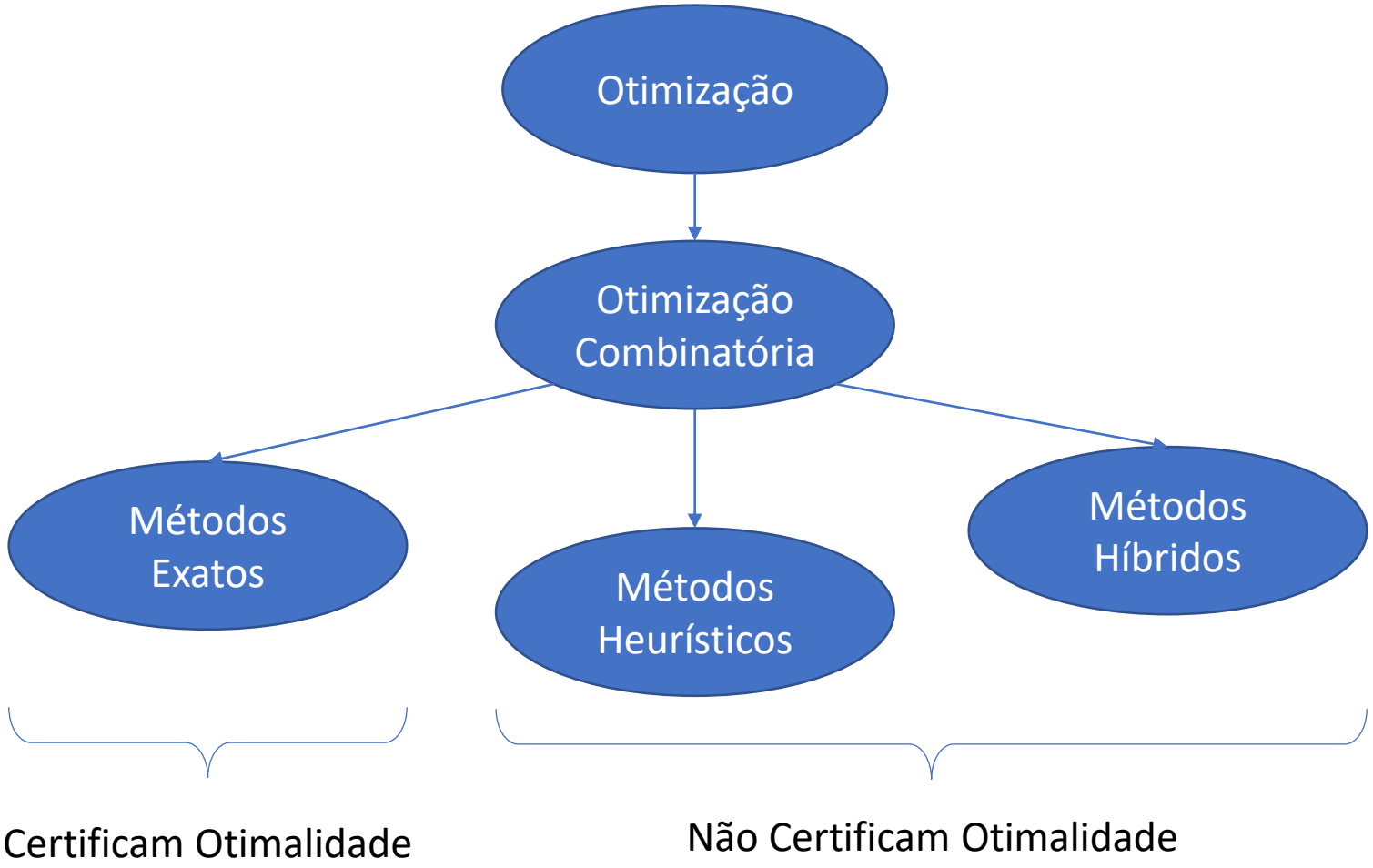
Métodos Híbridos em Otimização Combinatória

Pedro Henrique González Silva
pegonzalez@cos.ufrj.br

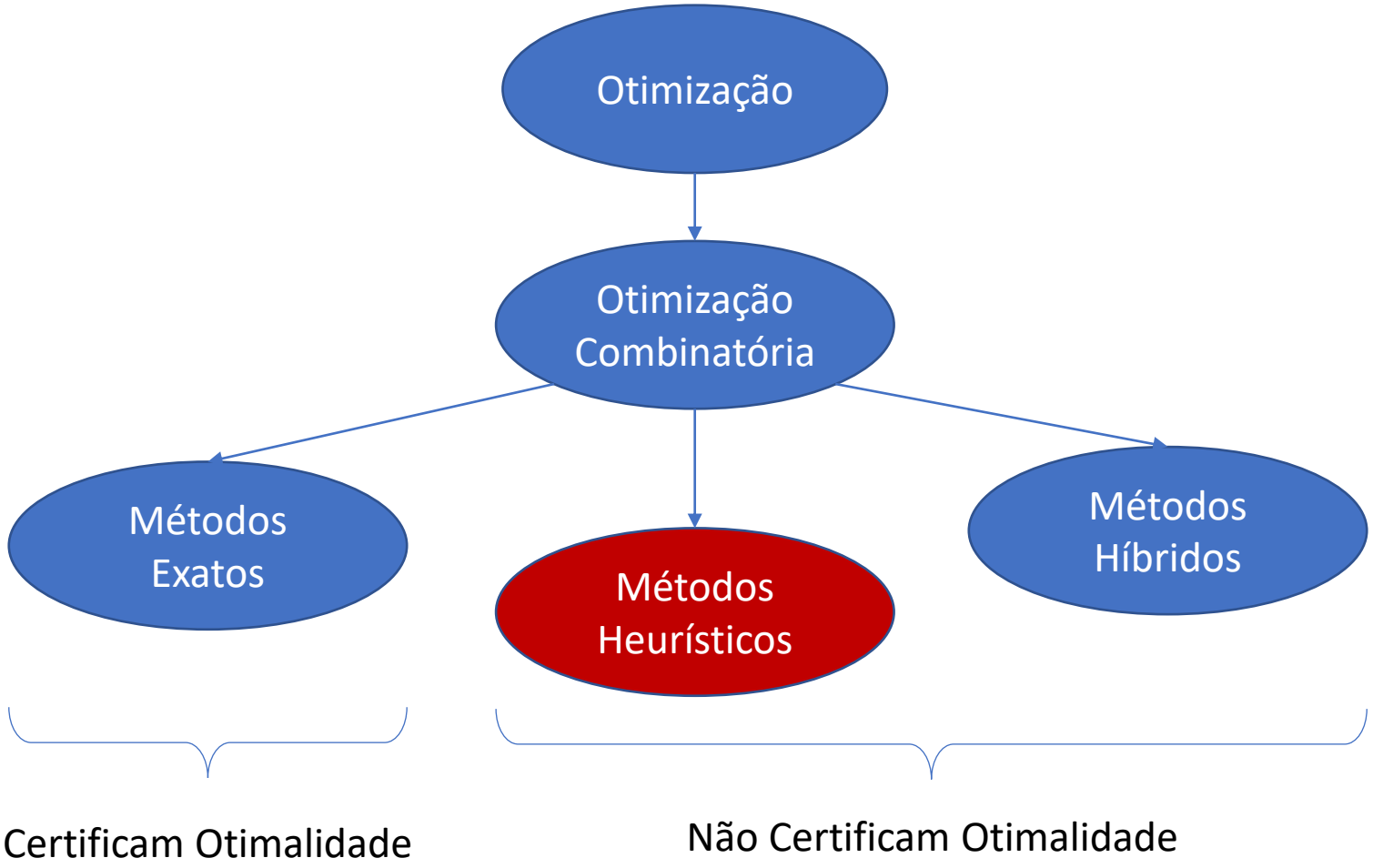
Contextualização



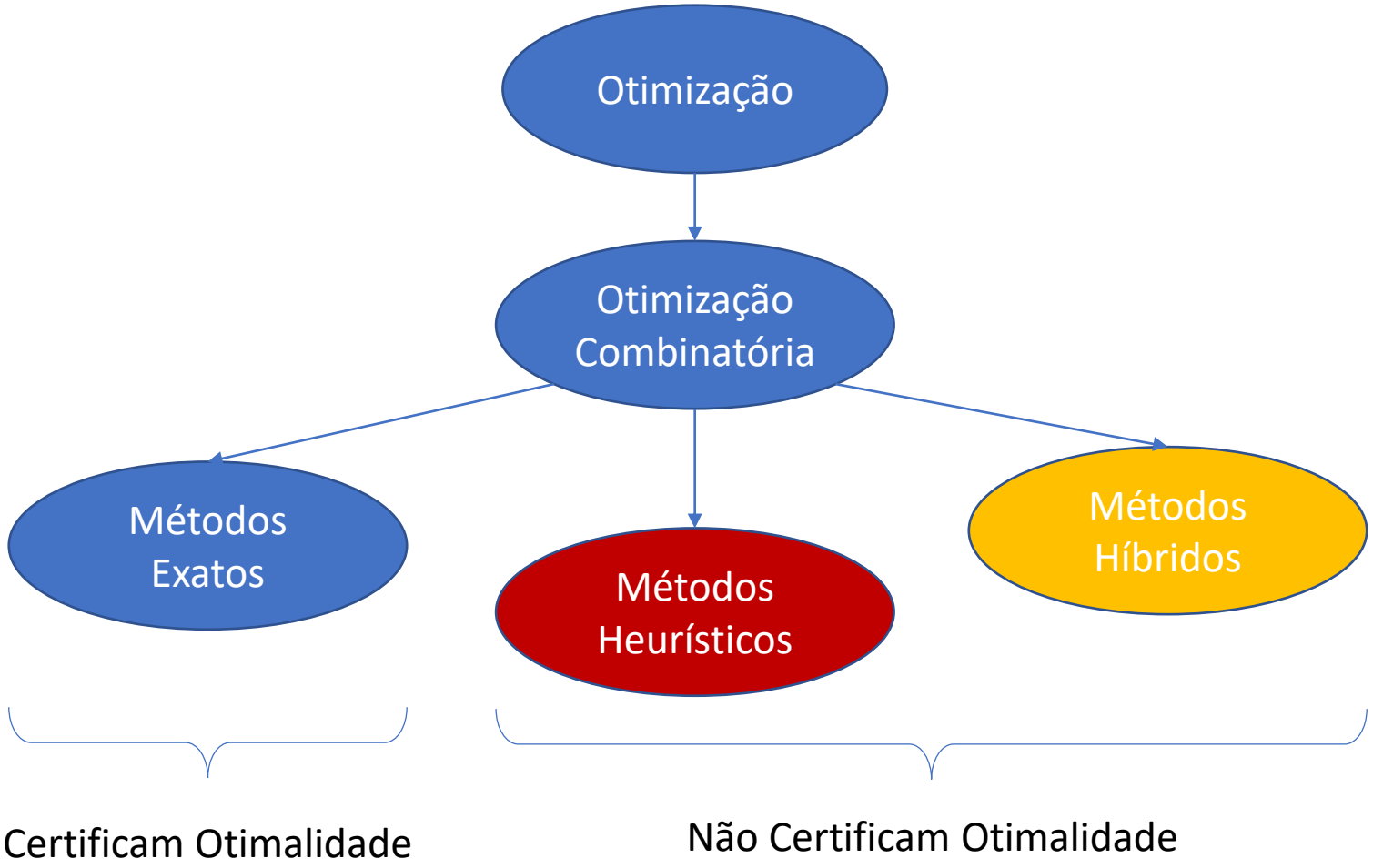
Contextualização



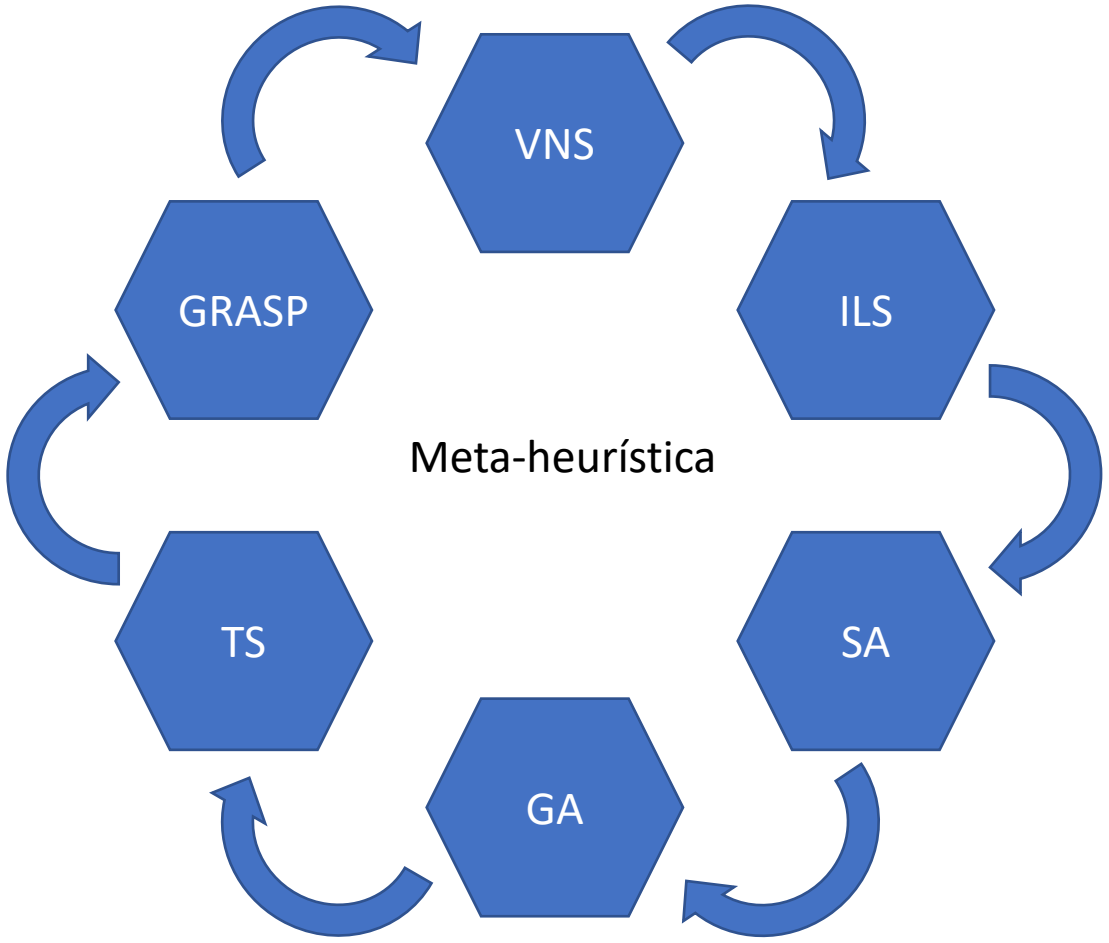
Contextualização



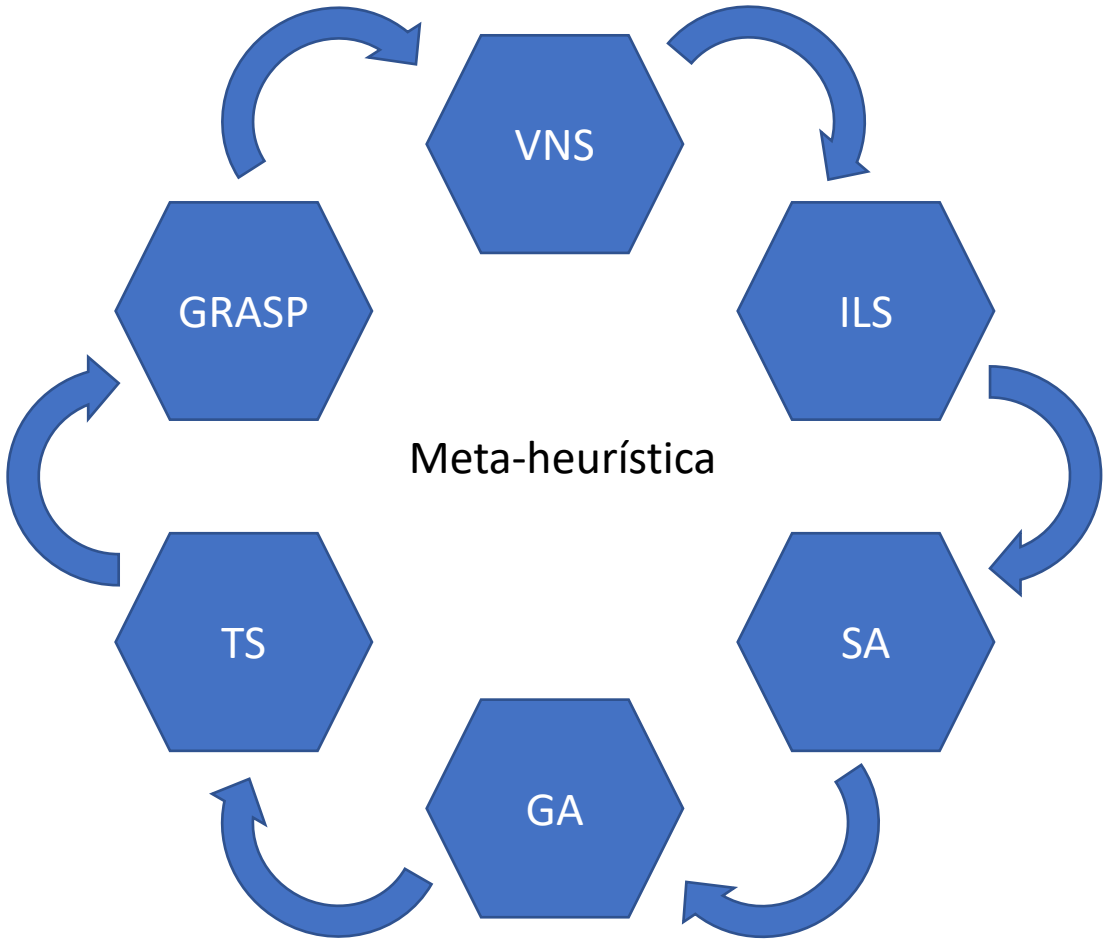
Contextualização



Contextualização

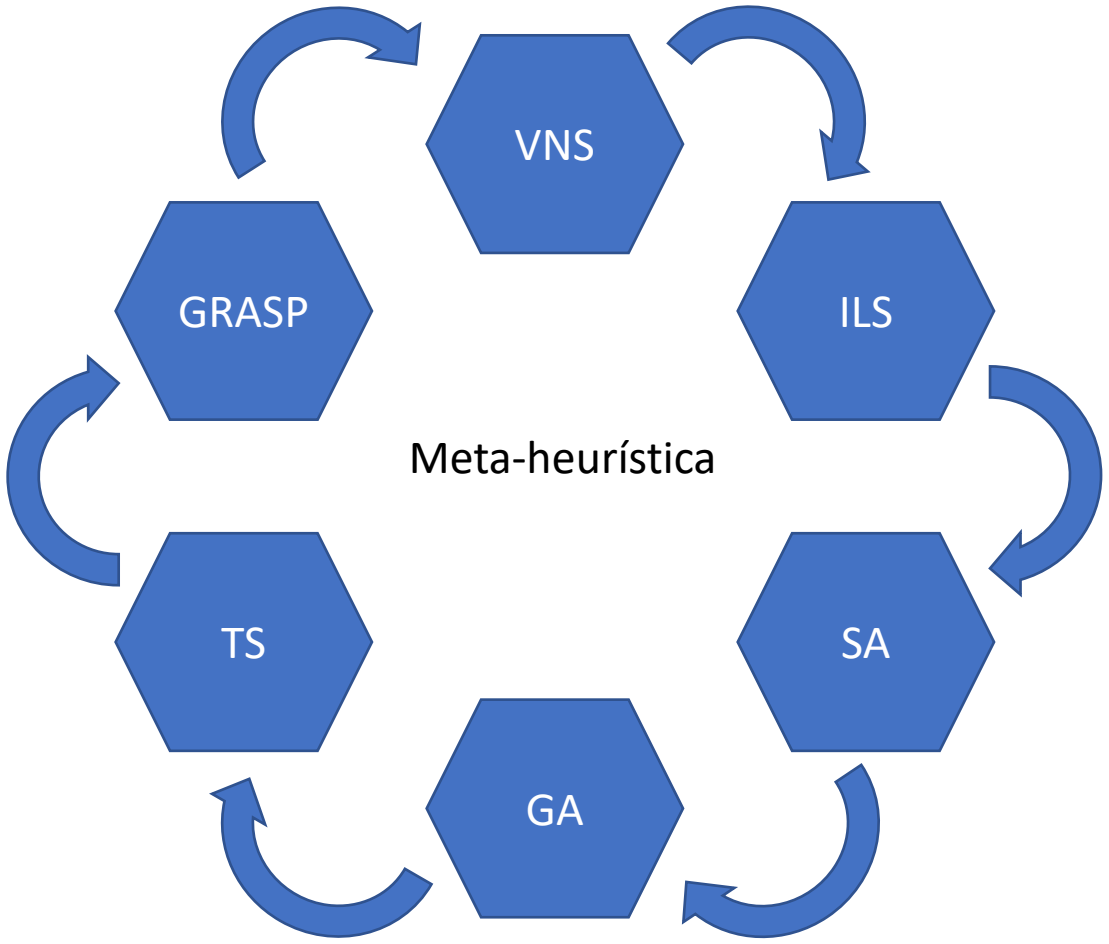


Contextualização



Como hibridizá-las ?

Contextualização



Como hibridizá-las ?

Heurísticas utilizando MIP

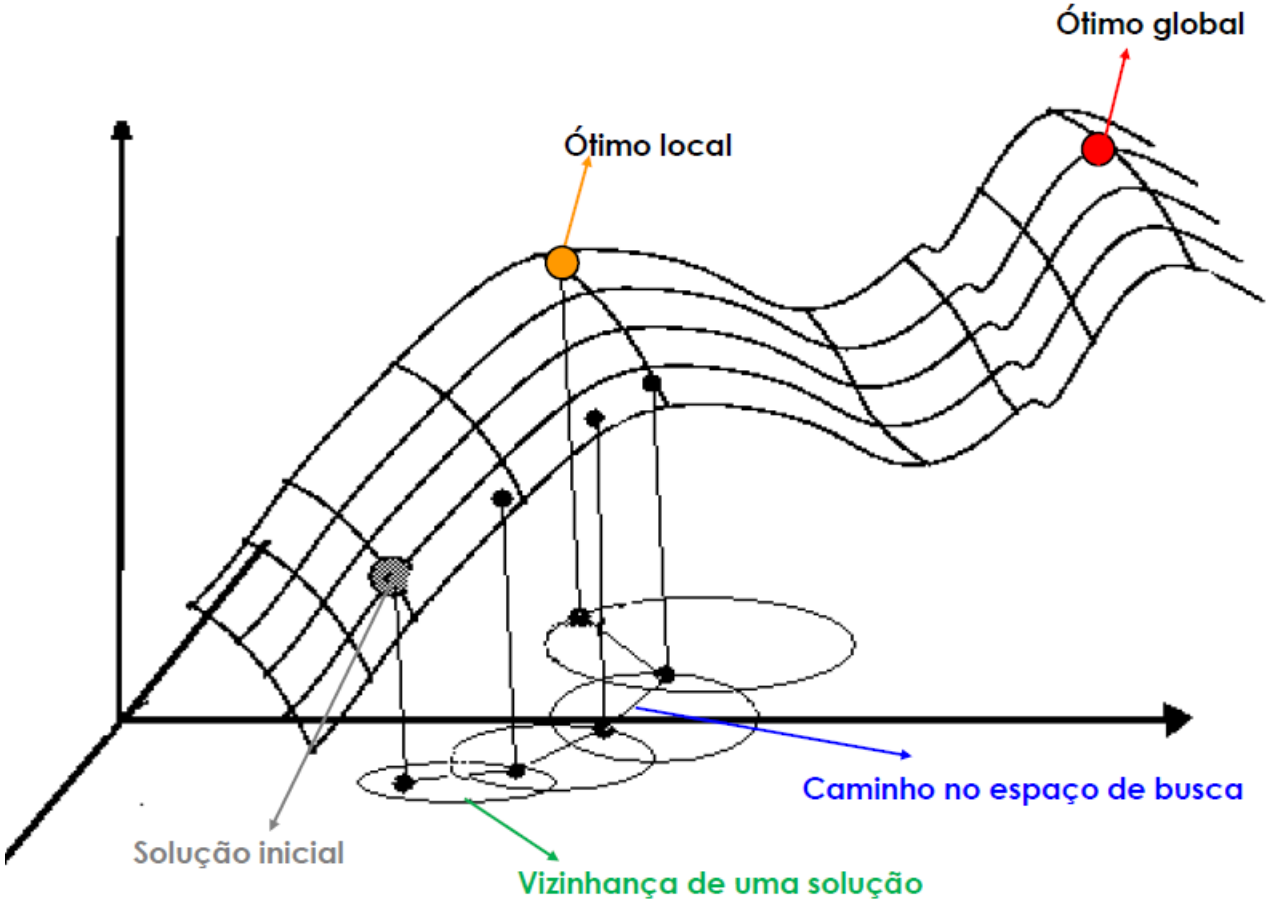
Revisão

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurísticas
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

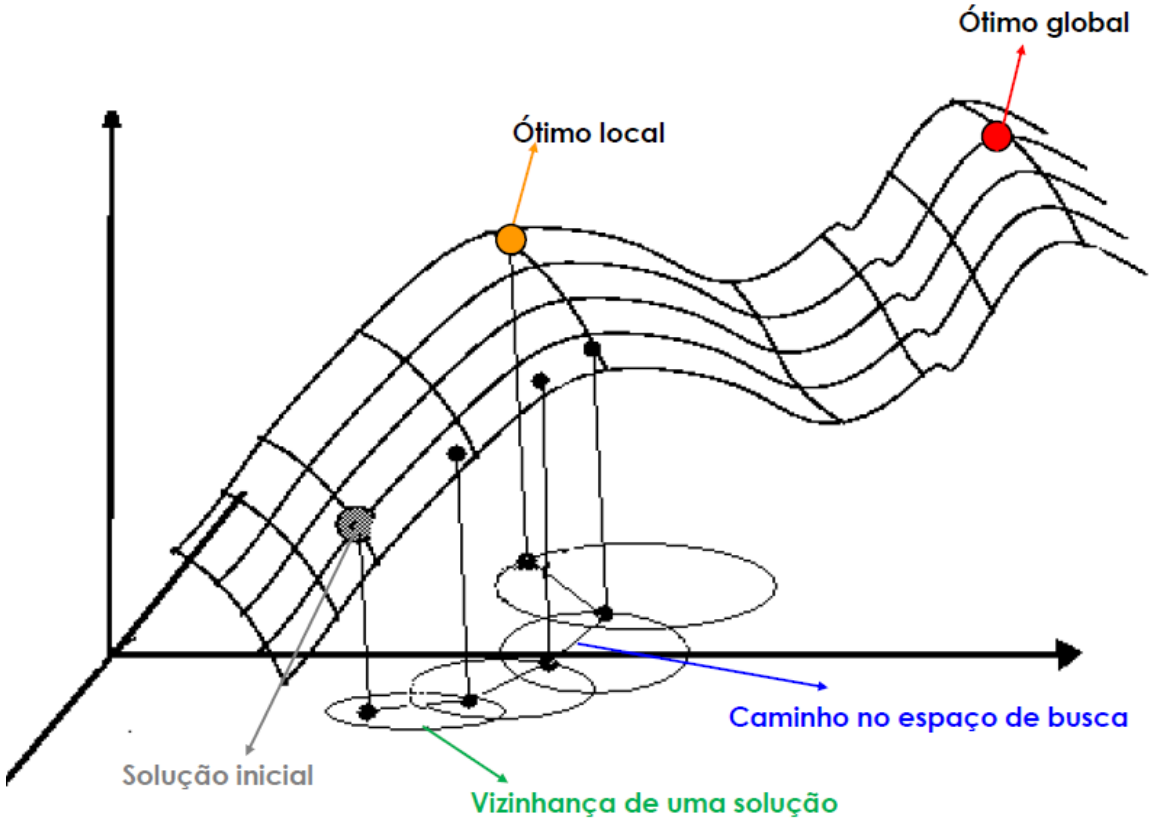
Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurísticas
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Introdução


Ótimo global
Ótimo local
Caminho no espaço de busca
Solução inicial
Vizinhança de uma solução

Introdução



Conceitos Chave

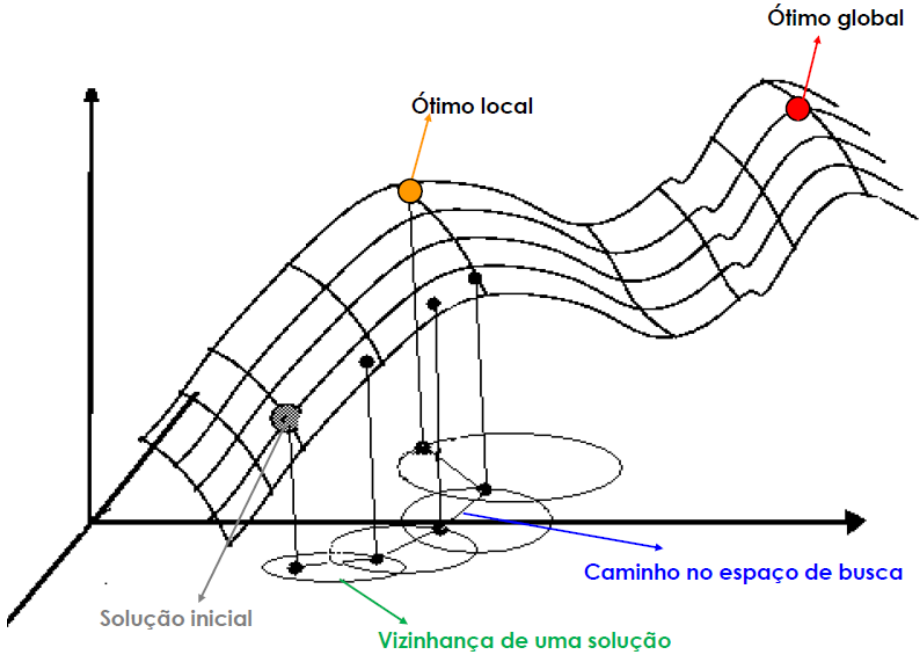
- Solução (inicial)
- Vizinhança de uma solução
- Caminho no espaço de busca
- Ótimo local
- Ótimo global

Introdução

Solução e suas representações

- Seja dado um conjunto $S \subseteq 2^{|E|}$ de soluções definido por subconjuntos de um conjunto suporte E de elementos que satisfazem as condições do problema que busca-se resolver

- Uma solução $s \in S$ pode ser representada de diversas, mas precisa indicar quais elementos de E pertencem a s.



Introdução

Solução e suas representações

- Exemplo: Problema da Mochila 0-1

Cada item i possuí um peso w_i e um valor p_i





Capacidade da Mochila (W)

Introdução

Solução e suas representações

- Exemplo: Problema da Mochila 0-1
  - A representação de uma solução viável com n itens pode ser feita através de um vetor binário x com n posições, tal que:

$$\begin{cases} x_i = 1, \ quando\ o\ item\ i\ estiver\ na\ solução, 1 \leq i \leq n \\ x_i = 0, \ caso\ contrário, 1 \leq i \leq n \end{cases}$$

Exemplo para n=4: (0,0,1,0), (1,0,1,0), (1,1,0,0), (1,0,1,1)

  - Outra possível representação de uma solução viável com n itens pode ser feita através de um conjunto de inteiros, onde o item é representado no conjunto, caso pertença a solução.

Exemplo para n=4: {3}, {1,3}, {1,2}, {1,3,4}

Introdução

Solução e suas representações

- Exemplo: Problema da Mochila 0-1
  - A representação de uma solução viável com n itens pode ser feita através de um vetor binário x com n posições, tal que:

$$\begin{cases} x_i = 1, \ quando\ o\ item\ i\ estiver\ na\ solução, 1 \leq i \leq n \\ x_i = 0, \ caso\ contrário, 1 \leq i \leq n \end{cases}$$

Exemplo para n=4: (0,0,1,0), (1,0,1,0), (1,1,0,0), (1,0,1,1)

  - Outra possível representação de uma solução viável com n itens pode ser feita através de um conjunto de inteiros, onde o item é representado no conjunto, caso pertença a solução.

Exemplo para n=4: {3}, {1,3}, {1,2}, {1,3,4}

ATENÇÃO!!!!

Existem diversas formas de representar uma solução, sendo assim, a escolha de uma forma pode vir a ser mais interessante do que outra dependendo do que se pretende fazer computacionalmente.

Introdução

Solução e suas representações

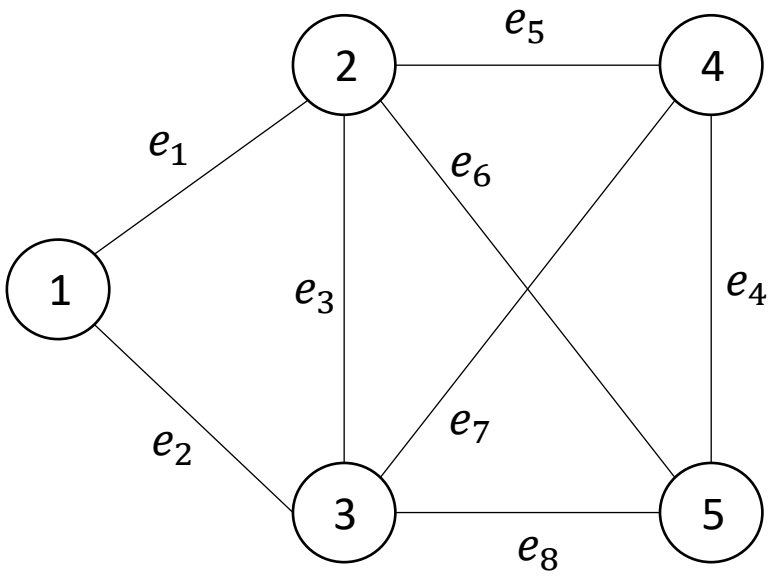
- Exemplo: Problema do Caixeiro Viajante

E: $Conjunto\ de\ arestas$
F: $Subconjunto\ de\ arestas\ de\ E\ que\ formam\ um\ ciclo\ Hamiltoniano$

Solução: vetor *v* com $m = |E|$ posições, tal que:

$$v_e = 1,\ se\ a\ aresta\ e\ pertence\ ao\ CH$$
$$v_e = 0,\ caso\ contrátio$$

Exemplo:
(1,1,0,1,1,0,0,1), (1,1,0,1,0,1,1,0)



Introdução

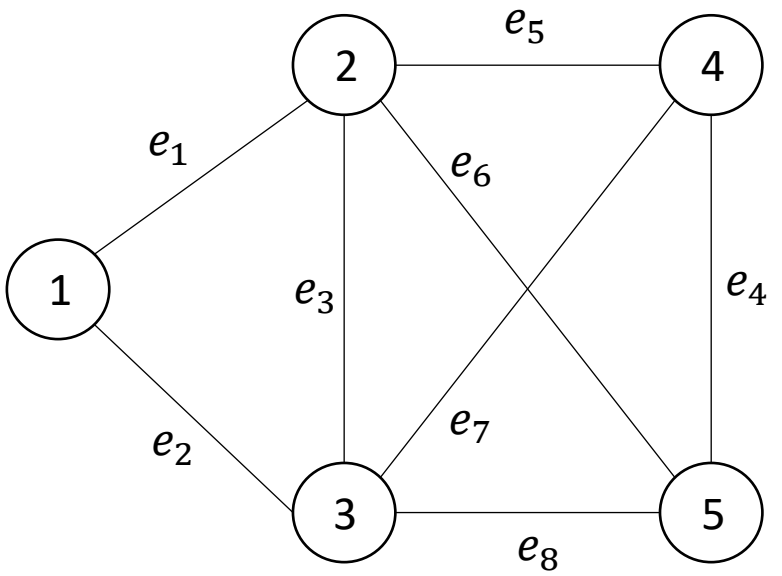
Solução e suas representações

- Exemplo: Problema do Caixeiro Viajante

  $E: Conjunto\ de\ arestas$
  $F: Subconjunto\ de\ arestas\ de\ E\ que\ formam\ um\ ciclo\ Hamiltoniano$

  Solução: conjunto ordenado de vértices visitados:

  Exemplo:
  {1,2,4,5,3}, {2,4,5,3,1}, {5,3,1,2,4}



Introdução

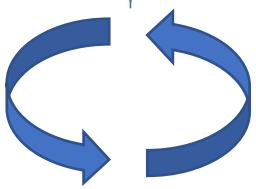
Solução e suas representações

- Exemplo: Problema do Caixeiro Viajante

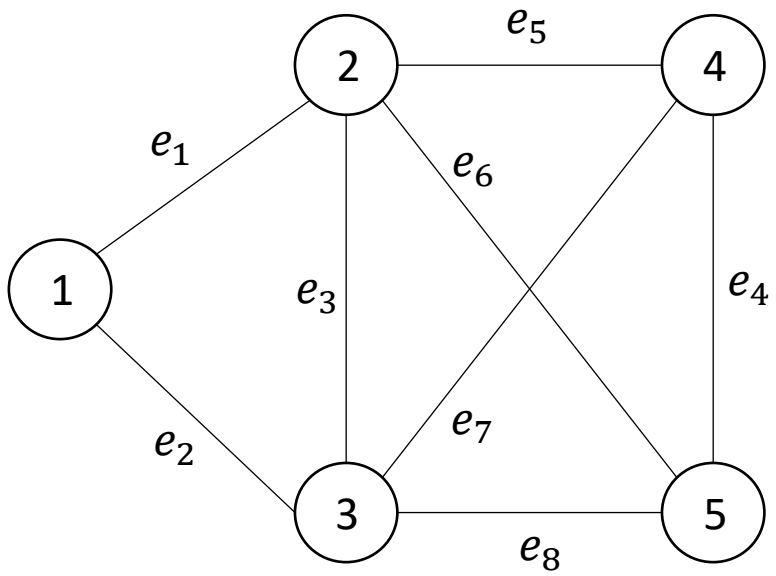
E: $Conjunto\ de\ arestas$
F: $Subconjunto\ de\ arestas\ de\ E\ que\ formam\ um\ ciclo\ Hamiltoniano$

Solução: conjunto ordenado de vértices visitados:

Exemplo:
{1,2,4,5,3}, {2,4,5,3,1}, {5,3,1,2,4}



Permutação Circular



Introdução

Solução e suas representações

- Exemplo: Problema do Caixeiro Viajante

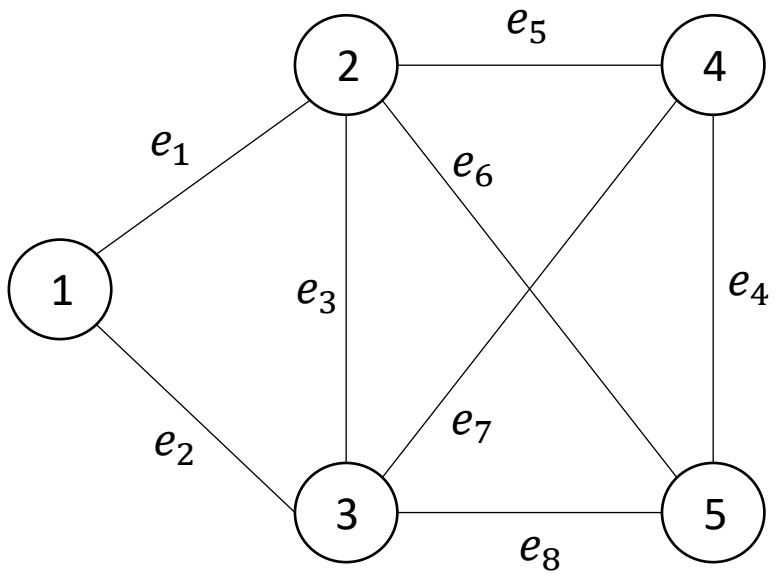
E: $Conjunto\ de\ arestas$
F: $Subconjunto\ de\ arestas\ de\ E\ que\ formam\ um\ ciclo\ Hamiltoniano$

Solução: conjunto ordenado de vértices visitados:

Exemplo:
{1,2,4,5,3}, {2,4,5 ,4}

SIMETRIA !!!

ermutação Circular



Introdução

Vizinhanças

- Problemas Combinatórios:
  - $f(s^*) = \min ou \max \{f(s): s \in S\}$, onde *S* é um conjunto discreto de Soluções.

- Vizinhança:
  - Elemento que permite representar a noção de proximidade entre as soluções em *S*.
  - Uma vizinhança de uma solução $s \in S$ é um mapeamento de S, tal que: $N: S \rightarrow 2^{|S|}$.

$$N(s) = \{s_1, s_2, \dots, s_k\}$$ **soluções vizinhas** de *s*

Notação: $N^k(s) = \{s_i: s_i \; está \; a \; uma \; distância \; k \; de \; s\}$

Introdução

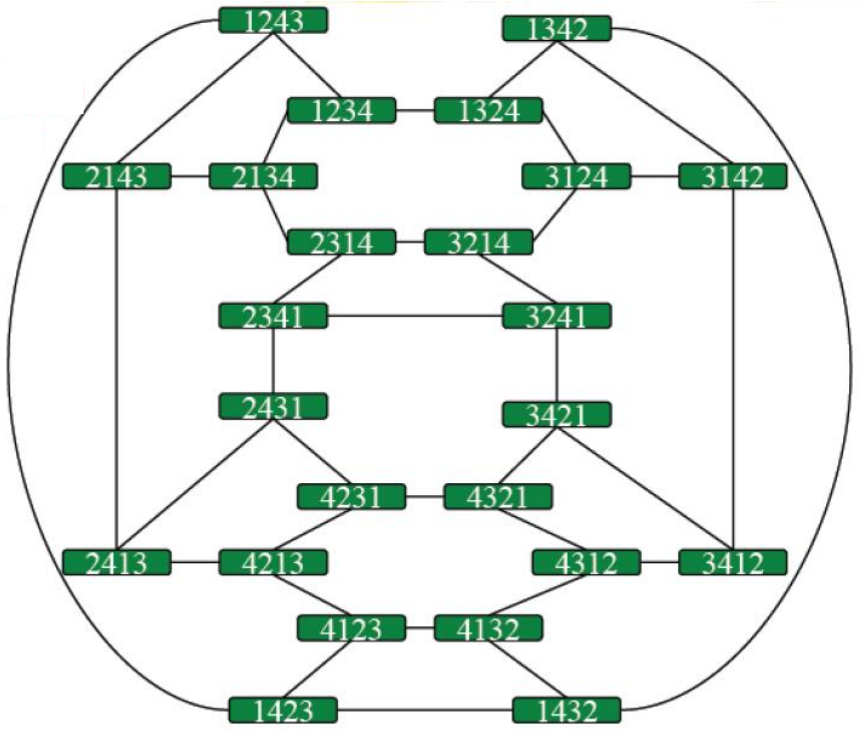
Vizinhanças

- Boas Vizinhança:
  - Devem ser compactas.
  - Devem representar um conjunto de soluções vizinhas.
  - Devem ser relacionadas a características interessantes do problema.
  - Seja possível explora-la de forma eficiente computacionalmente.

Introdução

Vizinhanças

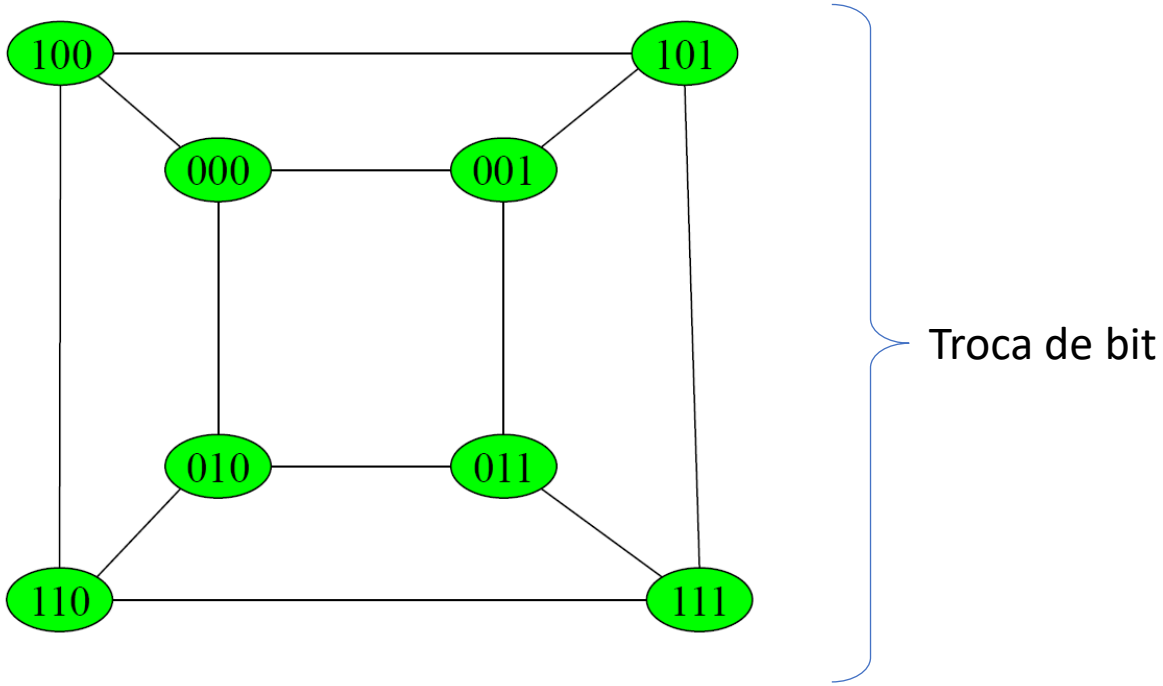
- Exemplos de Vizinhanças:
  - Vizinhanças no Espaço das Permutações:



Introdução

Vizinhanças

- Exemplos de Vizinhanças:
  - Vizinhanças no Espaço dos Vetores Binários de 3 Dimensões:



Introdução

Espaço de Busca

- Um espaço de busca é definido utilizando um conjunto solução *S* e uma vizinhança *N*.

- Como visto nos exemplos de vizinhanças, o espaço de busca pode ser representado por grafos onde cada vértice representa uma solução e cada aresta entre pares de vértice indica que se as soluções são (em caso de existência da aresta) ou não vizinhos (caso contrario).

- Um caminho no espaço de busca é formado por uma sequência de soluções, em que duas soluções consecutivas quaisquer são vizinhas, ou seja, um caminho no grafo usado para representar o espaço de busca.

- Dado um espaço de busca, ao definir a proximidade entre soluções (vizinhas ou não), naturalmente cria-se o conceito de distância entre soluções (espaço topológico)

Introdução

Ótimos Locais e Ótimos Globais

- Ótimo Local: solução tão boa ou melhor do que qualquer uma das soluções vizinhas. Para um problema de maximização:

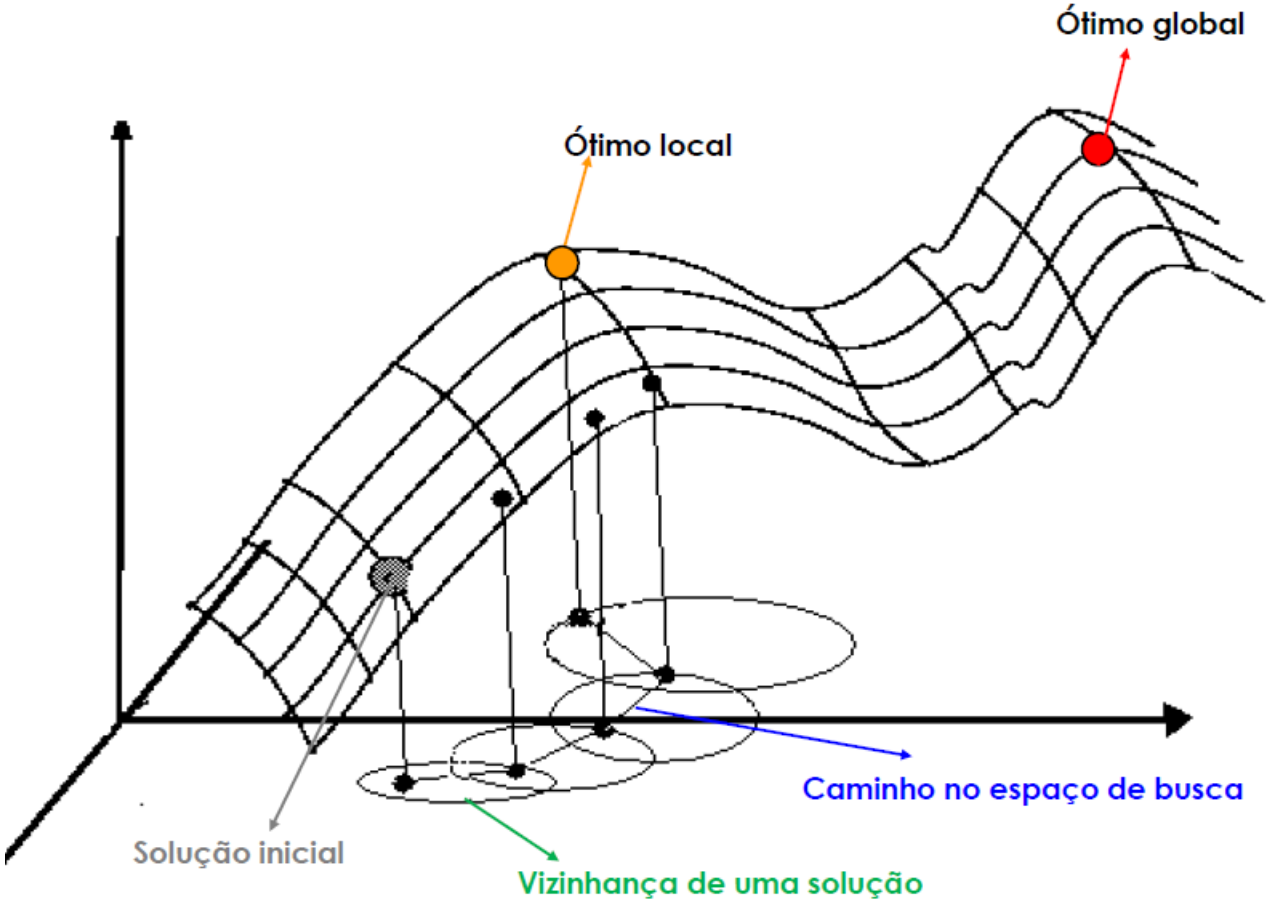
$$s^+ \text{ é ótimo local } \Leftrightarrow f(s^+) \geq f(s), \forall s \in N(s^+)$$

- Ótimo Global (s^*):

$$f(s^*) \geq f(s), \forall s \in S$$

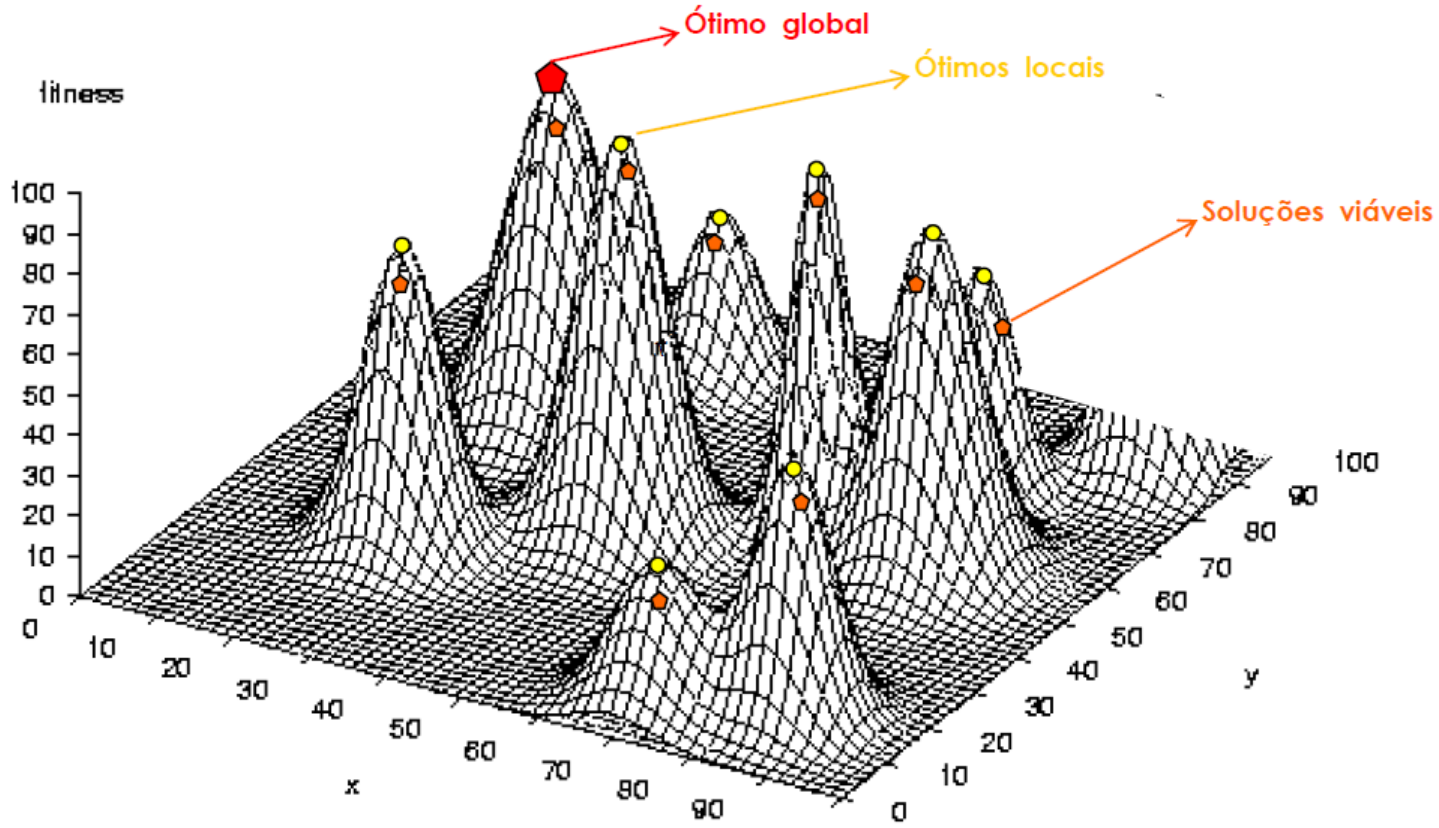
Introdução

Exemplo Superfície 1



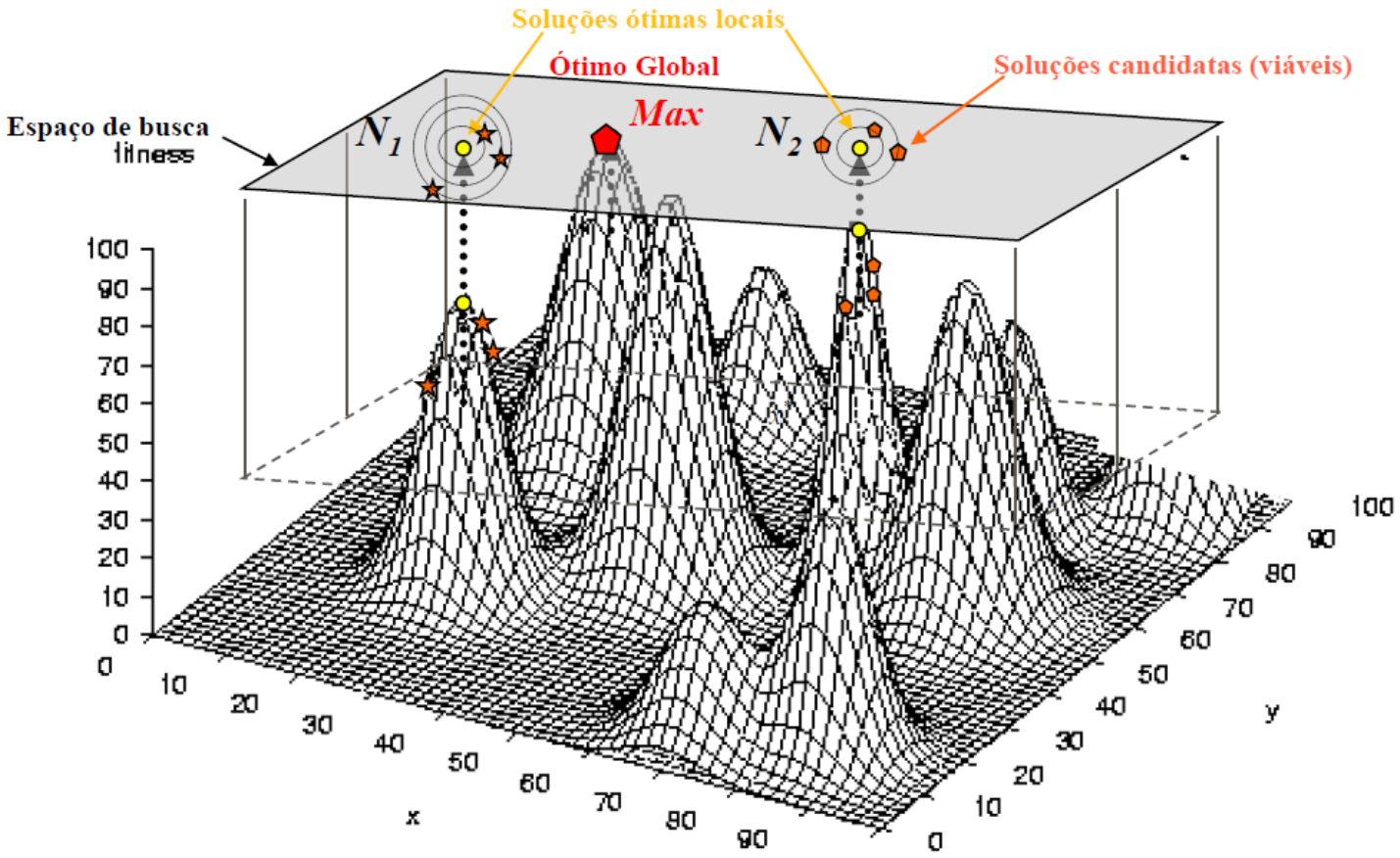
Introdução

Exemplo Superfície 2



Introdução

Exemplo Superfície 2



Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Algoritmos Construtivos

Introdução

Algoritmos construtivos, são heurísticas que buscam gerar uma solução que atenda todas as restrições do problema (solução viável), sem, necessariamente, maximizar ou minimizar a função a ser otimizada.

Algoritmos Construtivos

Introdução

Algoritmos construtivos, são heurísticas que buscam gerar uma solução que atenda todas as restrições do problema (solução viável), sem, necessariamente, maximizar ou minimizar a função a ser otimizada, mas buscando isso.



Algoritmos Construtivos

Hipótese e subestrutura ótima

- Os **algoritmos gulosos** são muito aplicados a problemas de otimização combinatória para a **obtenção de soluções iniciais** (ou seja, como **heurísticas construtivas**).

- Para que um **problema** seja solucionável por um algoritmo guloso é necessário que ele exiba uma **subestrutura ótima**, assim como na **programação dinâmica**.

- **Subestrutura ótima**: um problema possui uma subestrutura ótima quando é possível construir sua solução ótima a partir de soluções ótimas de partes do problema.

- Em outras palavras, um problema possui uma subestrutura ótima se a sua solução ótima contém soluções ótimas de subproblemas seus.

Algoritmos Construtivos

Miopia do Algoritmo Guloso

- Para resolver um problema, o **algoritmo guloso escolhe, a cada iteração, o objeto mais "apetitoso" que vê pela frente**. O objeto escolhido passa, então, a fazer parte da solução que o algoritmo constrói.

- **O algoritmo guloso é míope**: ele toma decisões com base nas informações disponíveis na iteração corrente, sem olhar as consequências que essas decisões terão no futuro. Um algoritmo guloso jamais se arrepende ou volta atrás: as escolhas que faz em cada iteração são definitivas.

- **Embora algoritmos gulosos pareçam obviamente corretos, a prova de sua corretude nem sempre é simples**. Para compensar, algoritmos gulosos são de fácil concepção, de fácil implementação, muito rápidos e eficientes.

- **Problemas que admitem soluções gulosas ótimas para qualquer instância de entrada são raros**. Na prática, os algoritmos gulosos raramente geram soluções ótimas, às vezes geram soluções muito boas, e às vezes geram soluções ruins. Exemplo de sucesso: Dijsktra para SPT(≥ 0); para Kruskal para AGM.

Algoritmos Construtivos

Preceitos e premissas

- Preceito: fazer uma escolha gulosa(míope), na esperança de que esta escolha leve a uma boa solução global (possivelmente ótima).

- Premissas – a escolha feita em cada passo deve ser:
  - Viável: satisfazer as restrições do problema;
  - Localmente ótima: a melhor escolha local dentre todas as escolhas viáveis; e
  - Irrevogável: uma vez feita, a escolha não pode ser alterada nos passos subsequentes (invariância).

Os algoritmos gulosos nem sempre encontram soluções ótimas!!!

Algoritmos Construtivos

Exemplo Problema da Mochila 0-1

Cada item i possuí um peso w_i e um valor p_i

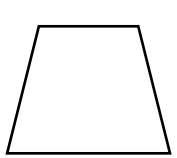




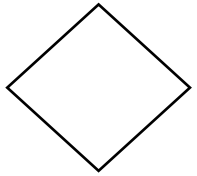
Capacidade da Mochila (W)

Algoritmos Construtivos

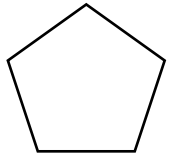
Exemplo Problema da Mochila 0-1



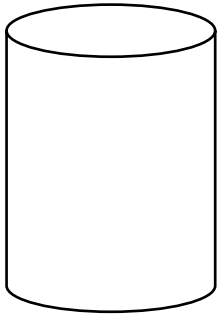
Peso: 10
Valor: 60



Peso: 20
Valor: 100



Peso: 30
Valor: 120



Capacidade: 50

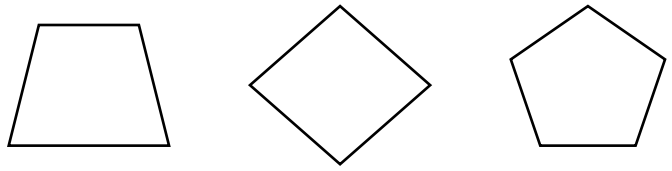


Algoritmos Construtivos

Exemplo Problema da Mochila 0-1

1. Calcular a razão $(\frac{p_i}{w_i})$ para cada item;
2. Ordenar os itens em função dessa razão;
3. Selecionar os itens com a maior razão até que o próximo item não possa ser adicionado em sua integralidade;

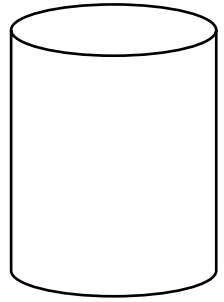




Peso: 10
Valor: 60

Peso: 20
Valor: 100

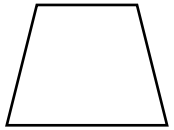
Peso: 30
Valor: 120



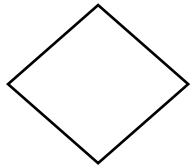
Capacidade: 50

Algoritmos Construtivos

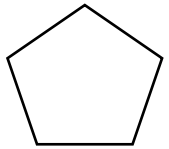
Exemplo Problema da Mochila 0-1



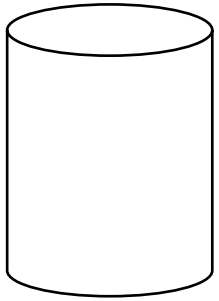
Peso: 10
Valor: 60



Peso: 20
Valor: 100



Peso: 30
Valor: 120



Capacidade: 50

Algoritmos Construtivos

Exemplo Caixeiro Viajante Simétrico

- Um conjunto de n cidades
- Uma matriz simétrica de distâncias
- Objetivo: Determinar a ordem de visitação de todas as cidades que resulte no somatório das menores distancias.



Dois algoritmos construtivos:

1. Vizinho Mais Próximo
2. Inserção Mais Barata

Algoritmos Construtivos

Exemplo Caixeiro Viajante Simétrico

- Considere o problema de CVS com n=5 e que o caixeiro mora na cidade 2.

- Considere também a matriz de distâncias abaixo:

Dist	1	2	3	4	5
1	0	15	20	4	9
2	15	0	50	8	7
3	20	50	0	12	18
4	4	8	12	0	17
5	9	7	18	17	0



Algoritmos Construtivos

Exemplo Caixeiro Viajante Simétrico

Heurística do Vizinho Mais Próximo

1. Inicia-se uma rota vazia
2. Adiciona-se a cidade de origem à rota
3. Escolhe, dentre todas as cidades que ainda não foram visitadas, a cidade *i* que tiver a menor distância para a última cidade inserida na rota.
4. Adicione *i* à rota.
5. Caso exista cidade não visitada, retorne ao passo 3. Senão, encerre o algoritmo.



Dist	1	2	3	4	5
1	0	15	20	4	9
2	15	0	50	8	7
3	20	50	0	12	18
4	4	8	12	0	17
5	9	7	18	17	0

Algoritmos Construtivos

Exemplo Caixeiro Viajante Simétrico

Heurística da Inserção Mais Barata

1. Construir uma rota passo a passo, partindo de uma rota inicial envolvendo 3 cidades (obtidas por qualquer método).

2. Adicionar a cada passo, a cidade *k* (ainda não visitada) entre as cidades *i* e *j,* cujo custo de inserção (distância) seja o mais barato.



Dist	1	2	3	4	5
1	0	15	20	4	9
2	15	0	50	8	7
3	20	50	0	12	18
4	4	8	12	0	17
5	9	7	18	17	0

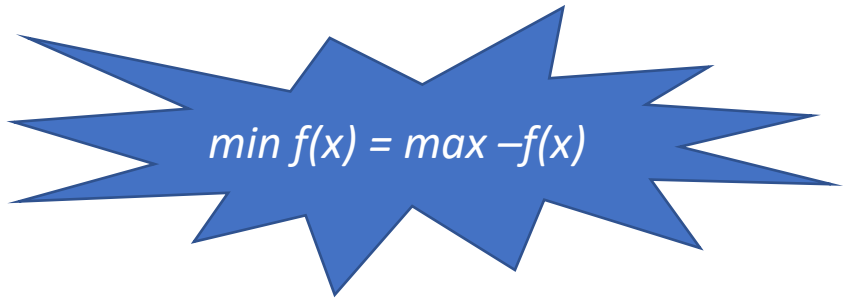
Algoritmos Construtivos

Lista de Candidatos Restrita

1. Seja c_i o critério guloso referente a inserção de um elemento *i* na solução corrente.
2. Define-se então $h_{min} = \min_i\{c_i\}$, $h_{max} = \max_i\{c_i\}$, $\alpha \in [0,1]$ e

$$LCR = \{i \mid h_{min} \leq c_i \leq h_{max} + \alpha(h_{min} - h_{max})\}$$

3. Uma vez tendo construído a LCR, escolhe-se aleatoriamente um elemento pertencente a ela.



Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

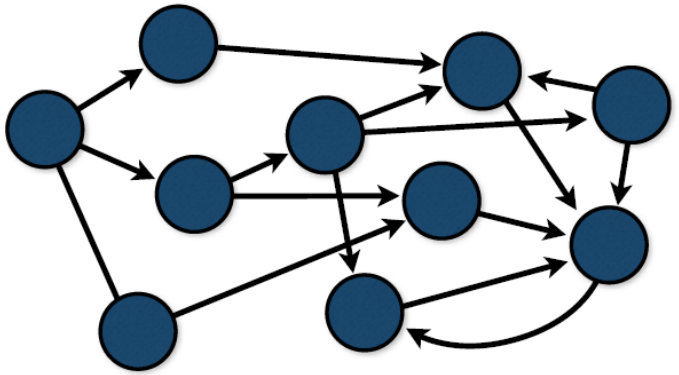
Algoritmos de Busca Local

Introdução

- São técnicas baseadas no conceito de vizinhança, e também são conhecidas como Heurísticas de Refinamento

- Movimento é definido como uma forma de modificação na solução corrente.

- **Definição**: uma heurística de refinamento/busca local é definida como um método que dada uma solução inicial, caminha de vizinho em vizinho de acordo com o movimento definido buscando melhorar a solução corrente, sem garantia de otimalidade.



Algoritmos de Busca Local

Introdução

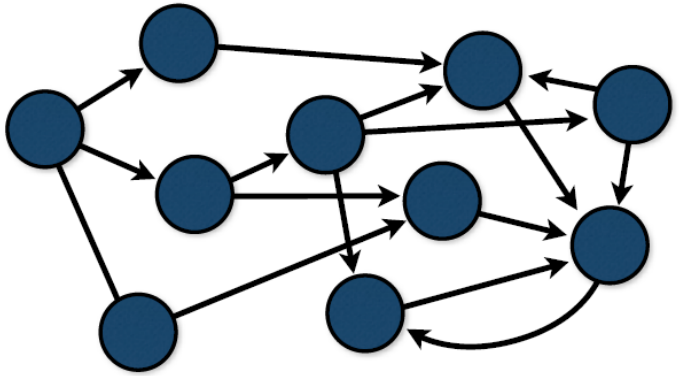
1. Solução Inicial

2. Definição da Vizinhança

3. Estratégia de Busca na vizinhança

    a) Melhor Aprimorante
    b) Primeiro Aprimorante

4. Complexidade de cada iteração
    1. Proporcional ao tamanho da vizinhaça
    2. Eficiência: Avaliação da função objetivo



Algoritmos de Busca Local

Introdução

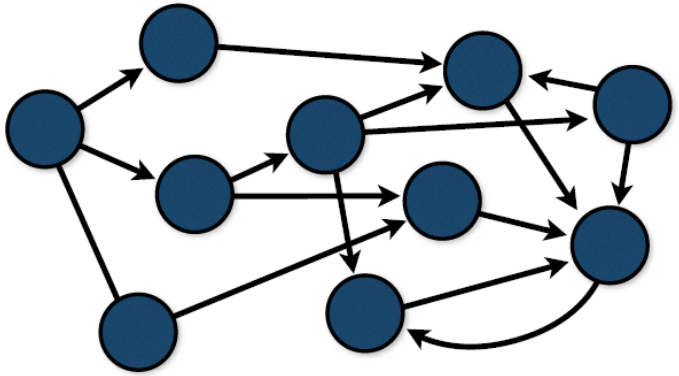
1. Solução Inicial

2. Definição da Vizinhança

3. Estratégia de Busca na vizinhança

    a) Melhor Aprimorante
    b) Primeiro Aprimorante

4. Complexidade de cada iteração
    1. Proporcional ao tamanho da vizinhança
    2. Eficiência: Avaliação da função objetivo



Uma busca local é dita eficiente se durante o processo, formos capazes de recalcular as variações na função objetivo de modo a atualizá-las apenas em relação às mudanças efetuadas na solução corrente, evitando cálculos repetitivos e desnecessários

Algoritmos de Busca Local

Melhor Aprimorante e Primeiro Aprimorante

Definição: São estratégias para determinar o quanto explorar da vizinhança

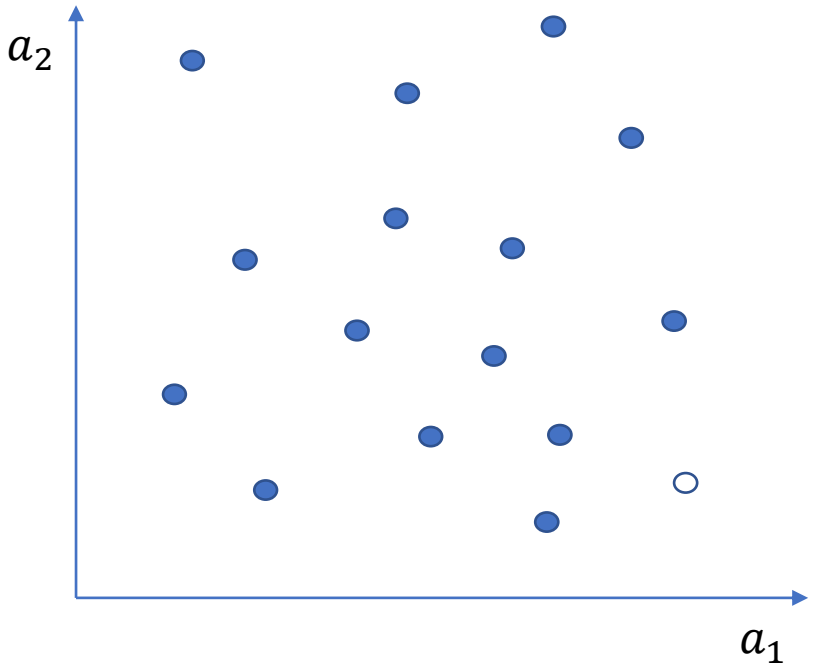
1. Melhor Aprimorante (*Best Improvement*)
    i. O movimento é realizado na direção da melhor solução na vizinhança, após explorar a vizinhança completamente.

2. Primeiro Aprimorante (*First Improvement*)
    i. O movimento é realizado na direção da primeira solução que melhorar a solução corrente, no momento que encontrá-la.

Algoritmos de Busca Local

Exemplo

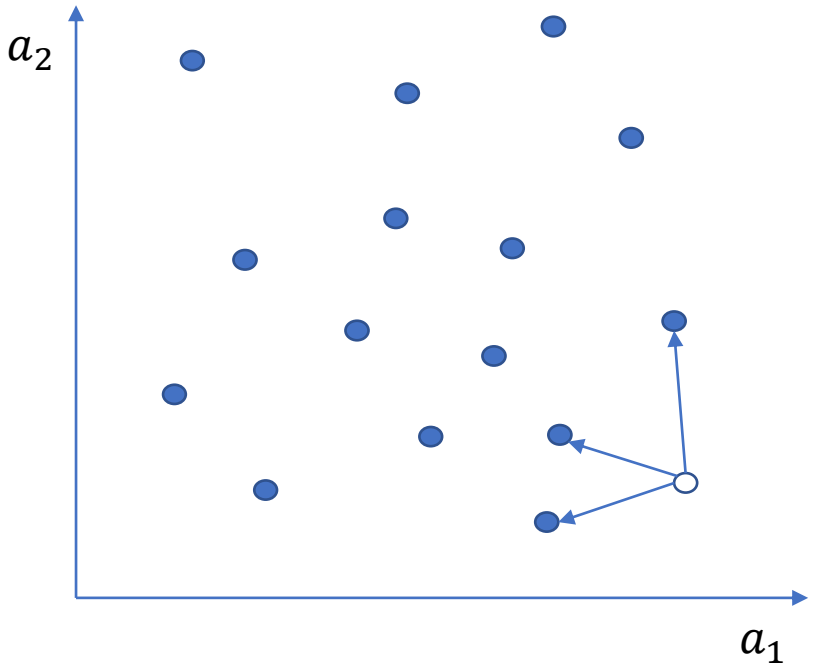
Solução Corrente



Algoritmos de Busca Local

Exemplo

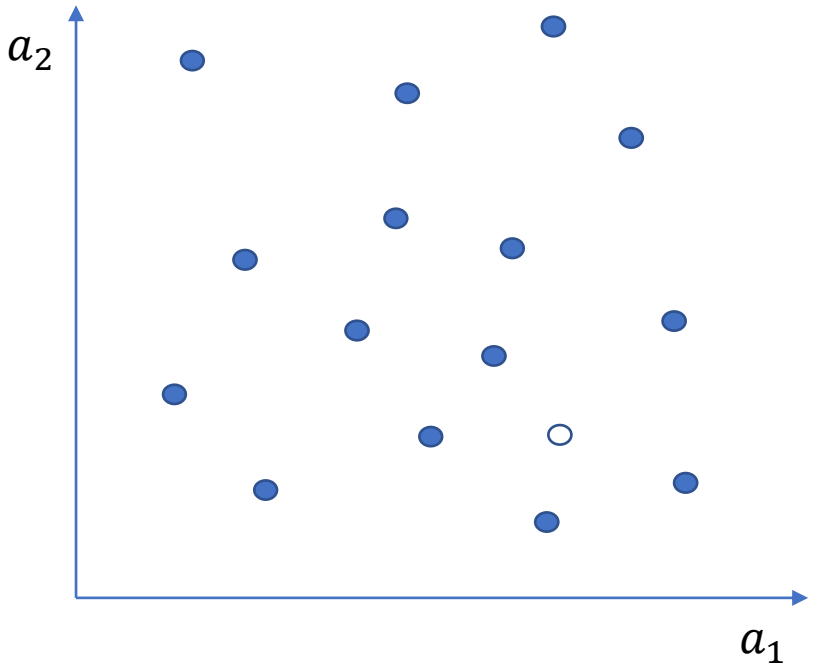
Solução Corrente



Algoritmos de Busca Local

Exemplo

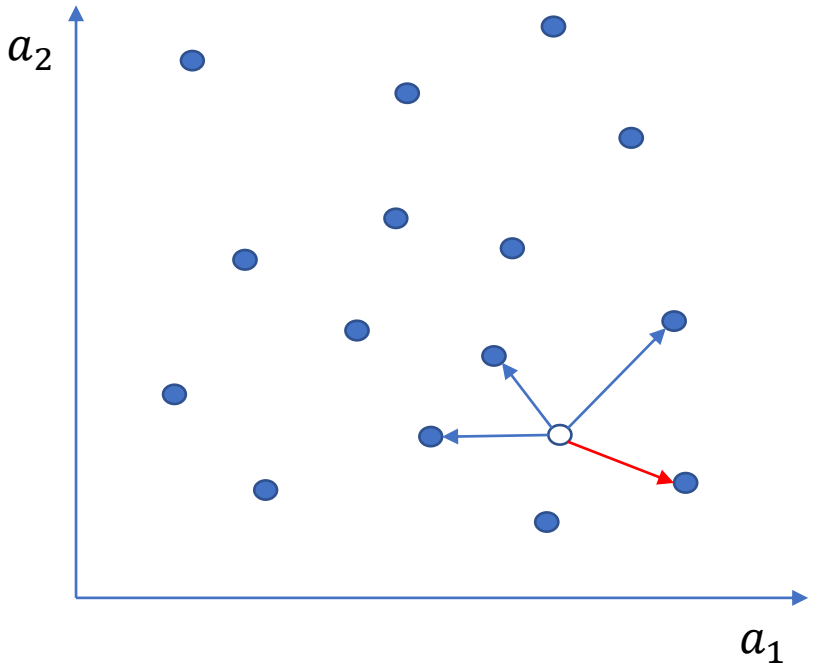
Solução Corrente



Algoritmos de Busca Local

Exemplo

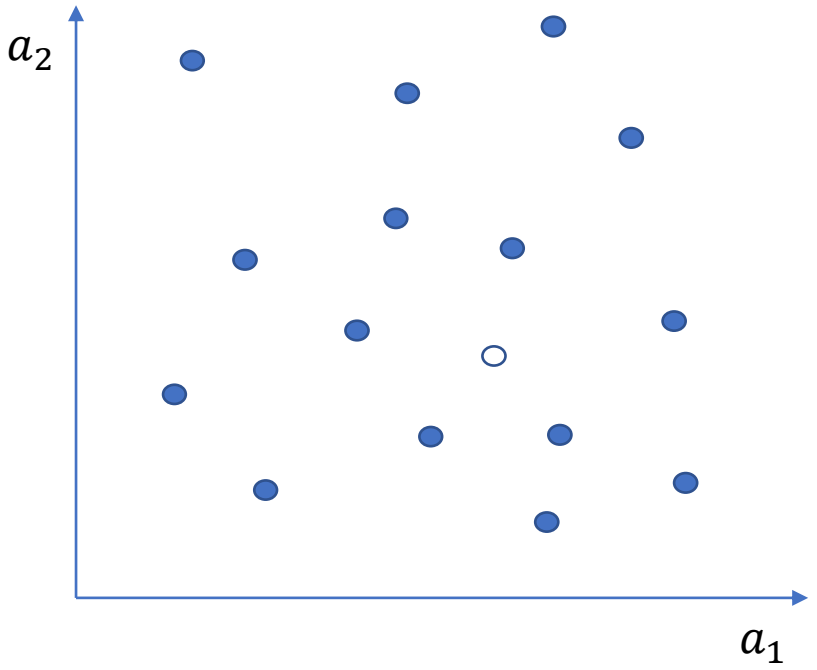
Solução Corrente



Algoritmos de Busca Local

Exemplo

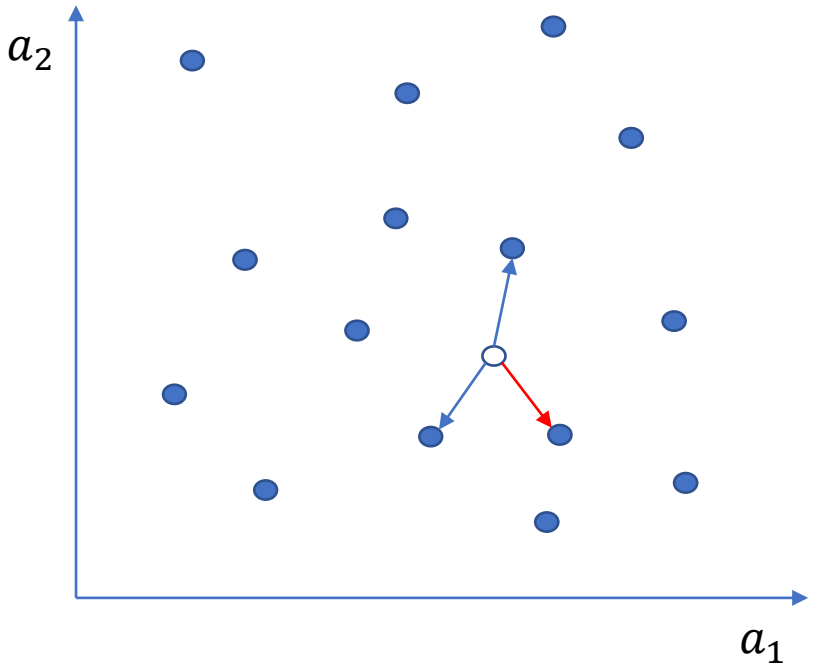
Solução Corrente



Algoritmos de Busca Local

Exemplo

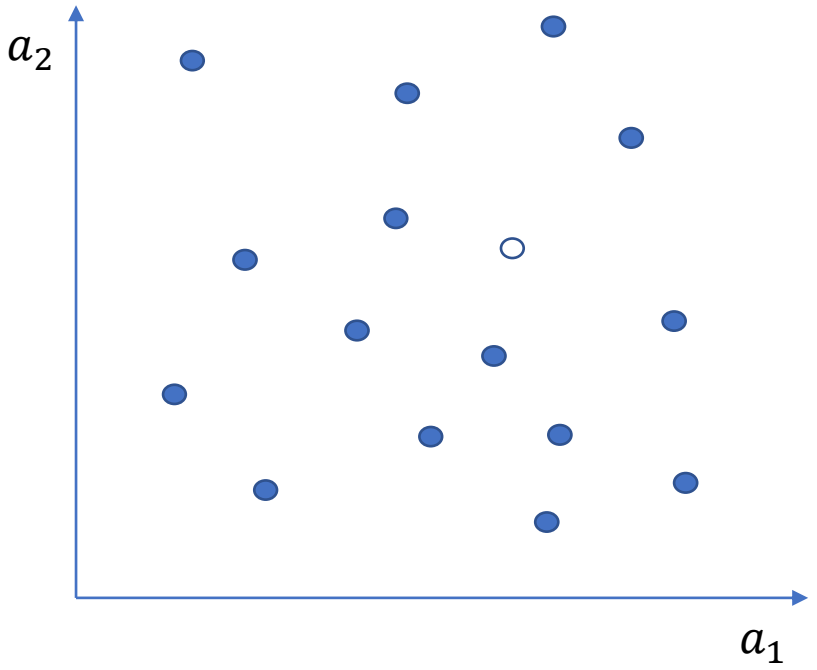
Solução Corrente



Algoritmos de Busca Local

Exemplo

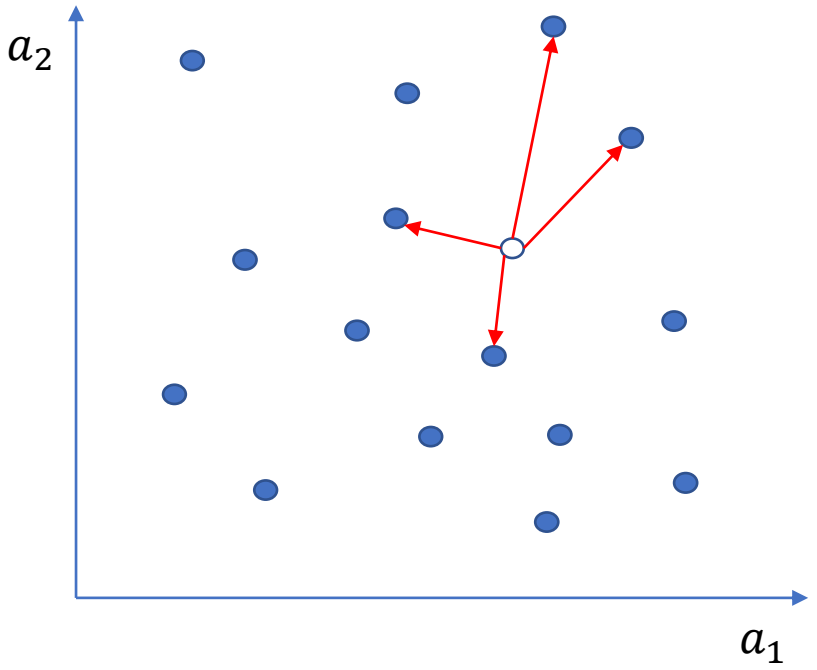
Solução Corrente



Algoritmos de Busca Local

Exemplo

Solução Corrente



Critério de Parada!!!

Algoritmos de Busca Local

Pseudocódigo – Minimização - Método da Descida

Selecione uma configuração inicial c;
$I = \{ n \in N(c) \mid f(n) < f(c) \}$;
while ($|I| > 0$) {
    c = selecionar uma configuração pertencente a I
    $I = \{ n \in N(c) \mid f(n) < f(c) \}$;
}
return c;

Algoritmos de Busca Local

Pseudocódigo – Minimização – Melhor Aprimorante

Selecione uma configuração inicial c;
$I = \{ n \in N(c) \mid f(n) < f(c) \}$;
while ($|I| > 0$) {
    c = selecionar uma configuração n pertencente a I tal que $f(n) \leq f(m) \; \forall \, m \in I$
    $I = \{ n \in N(c) \mid f(n) < f(c) \}$;
}
return c;

Algoritmos de Busca Local

Pseudocódigo – Minimização – Primeiro Aprimorante

```
Selecione uma configuração inicial c;
INICIO:
for (i ∈ N(c)) {
        if f(i) ≤ f(c){
                c = i;
                goto INICIO;
        }
}
return c;
```

Algoritmos de Busca Local

Variable Neighborhood Descent (VND)

Definição: É uma estratégia para combinação e uso sequencial de varias vizinhanças

```
Selecione uma configuração inicial c
k = 1
while (k ≤ r) {
        s = BuscaLocal(N_k(c), Tipo_Aprimorante, Melhora);
        if( Melhora == True) {
                k = 1;
                c = s;
        }else{
                k=k+1;
        }
}
return c;
```

Algoritmos de Busca Local

Random Variable Neighborhood Descendent (RVND)

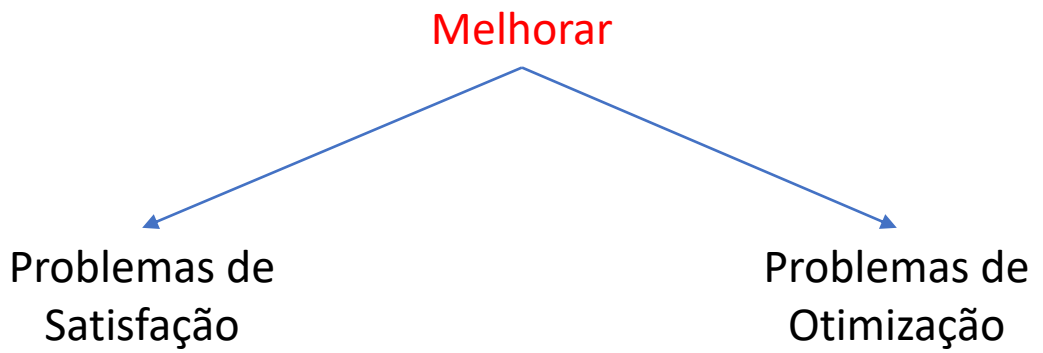
Definição: É uma estratégia para combinação e uso de varias vizinhanças

```
Selecione uma configuração inicial c
Definir uma permutação p[r] dos ids das vizinhanças
k = 1
while (k ≤ r) {
        s = BuscaLocal(N_p[k](c), Tipo_Aprimorante, Melhora);
        if( Melhora == True) {
                k = 1;
                c = s;
        }else{
                k=k+1;
        }

}
return c;
```

Algoritmos de Busca Local

Introdução



Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

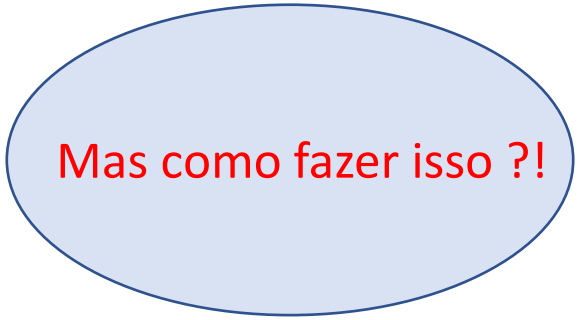
1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

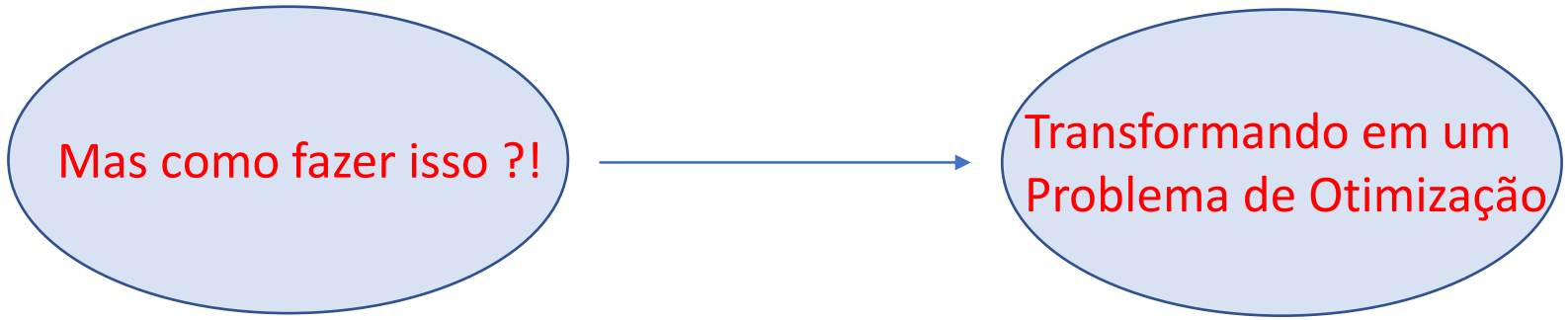


Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

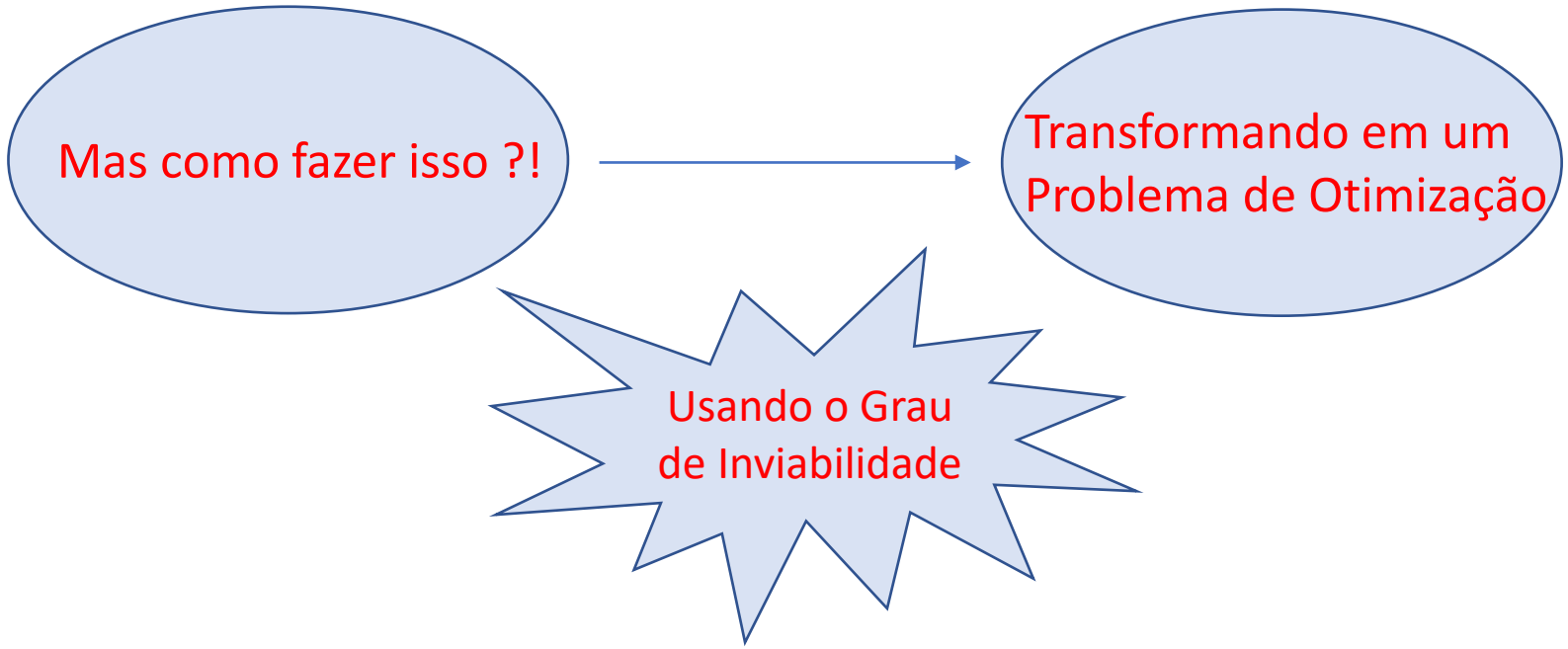


Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade



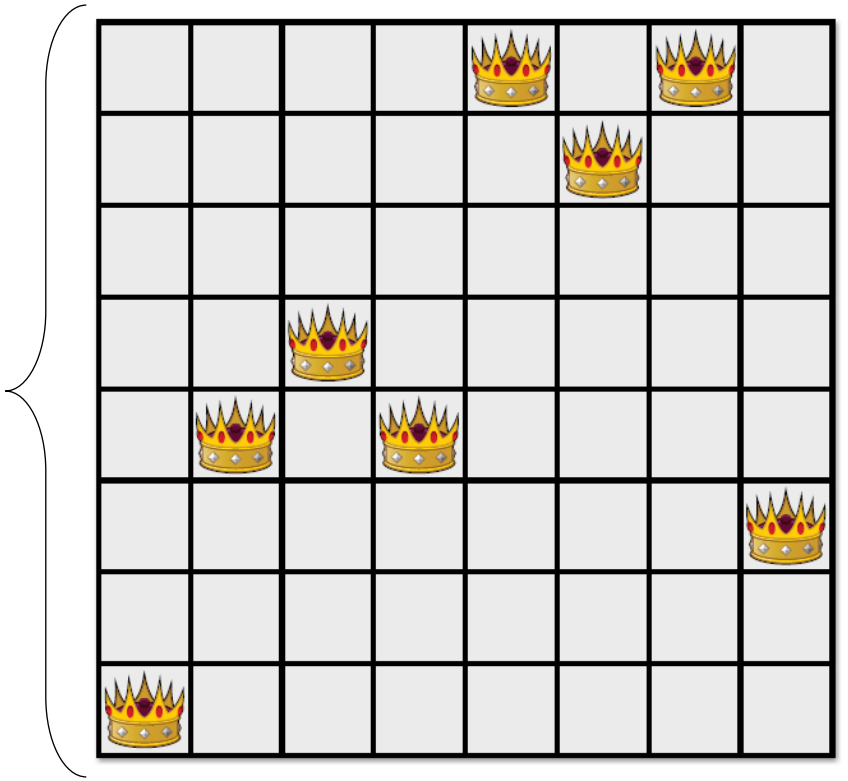
Algoritmos de Busca Local

Problema de Satisfação: Problema das n-rainhas

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



Algoritmos de Busca Local

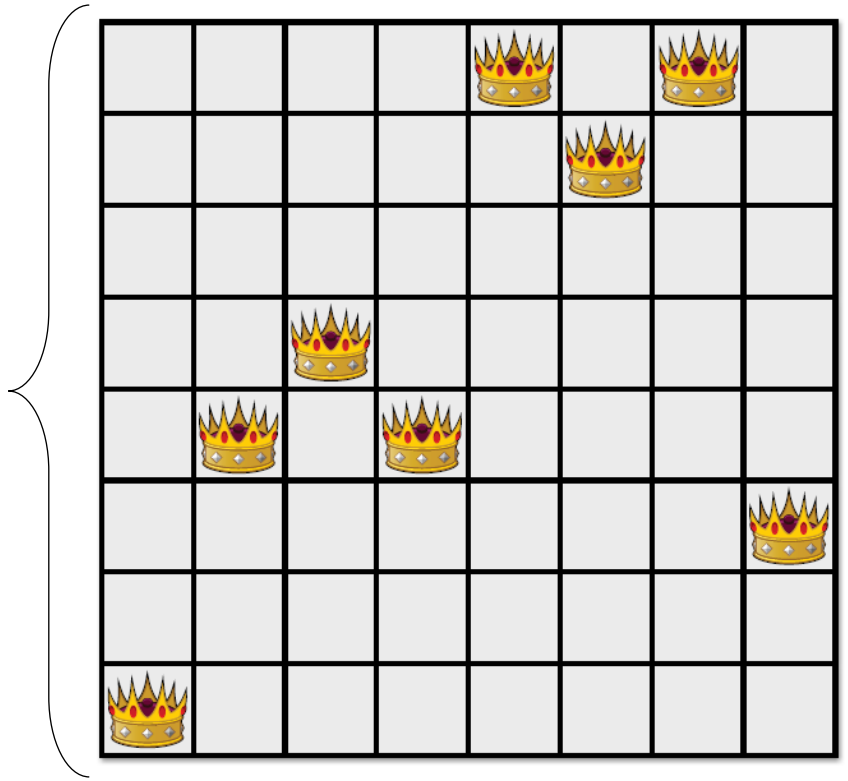
Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Qual seria uma vizinhança válida ?

Problema das n-rainhas



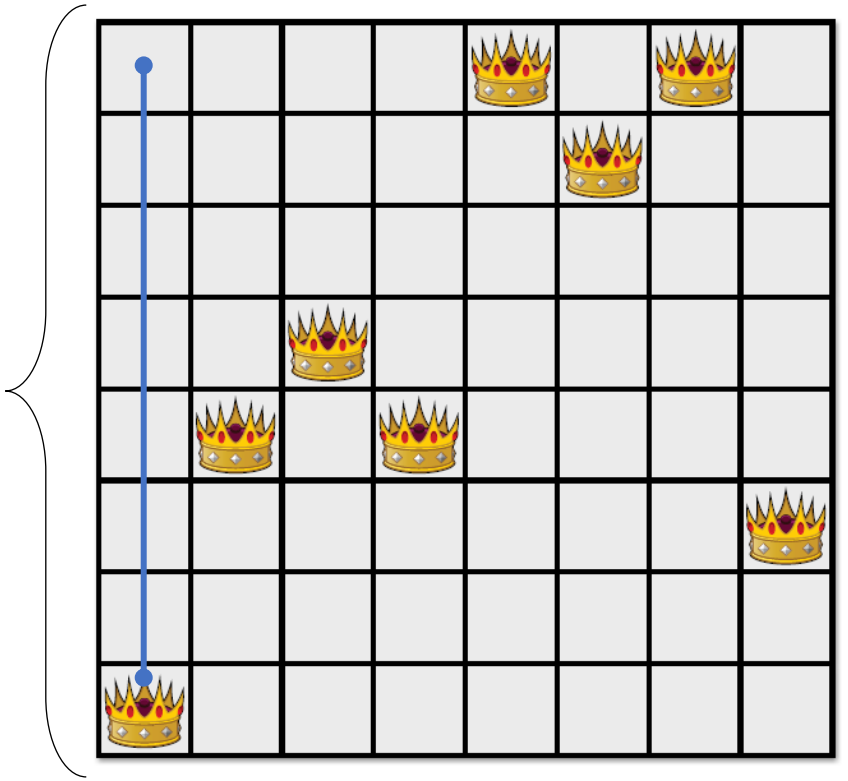
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



1

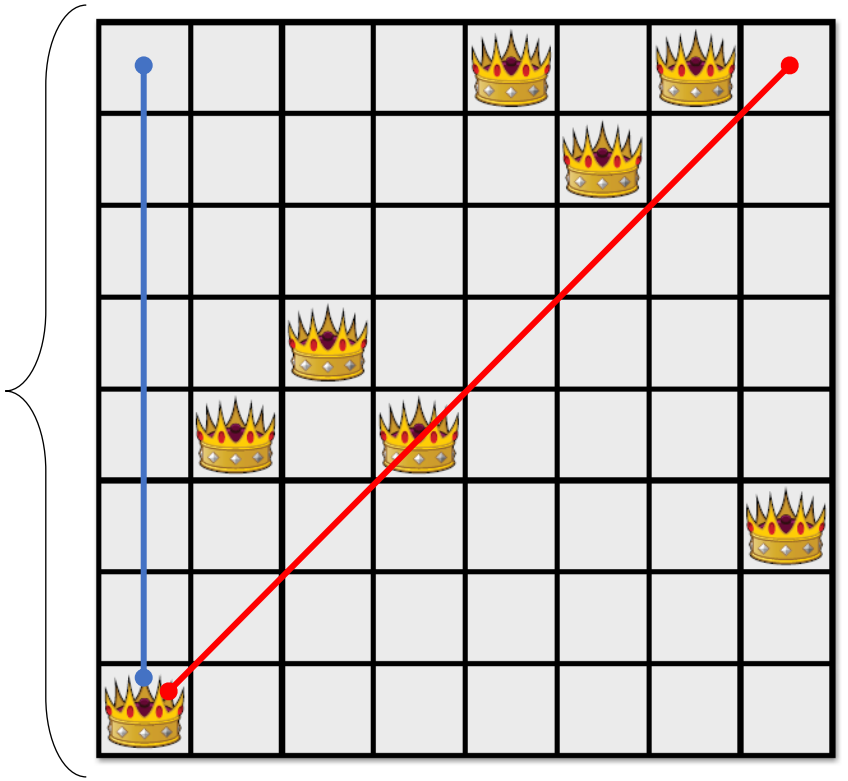
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



1

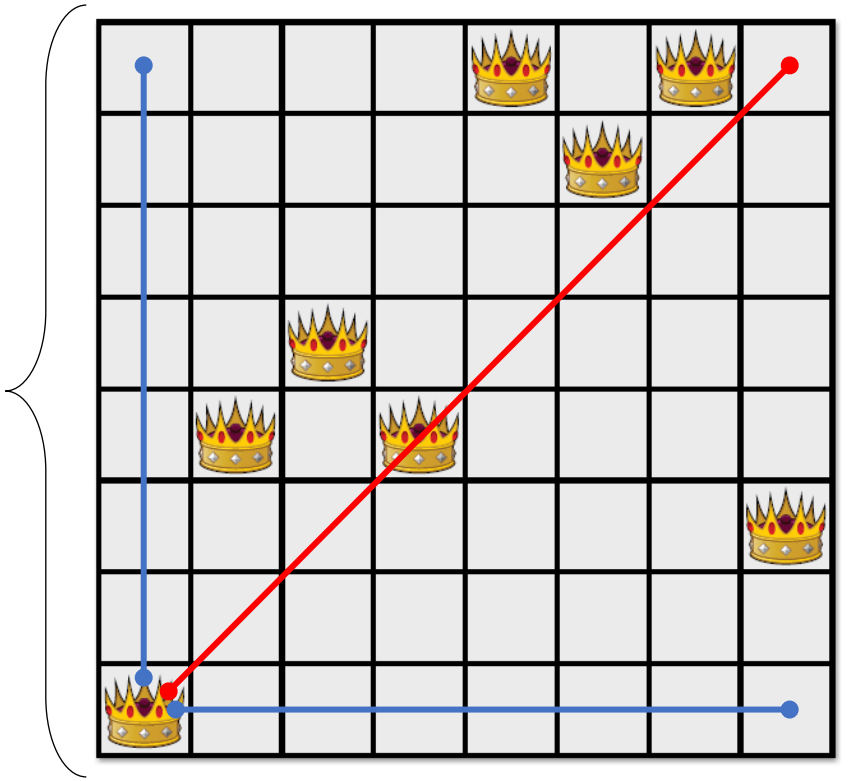
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



1

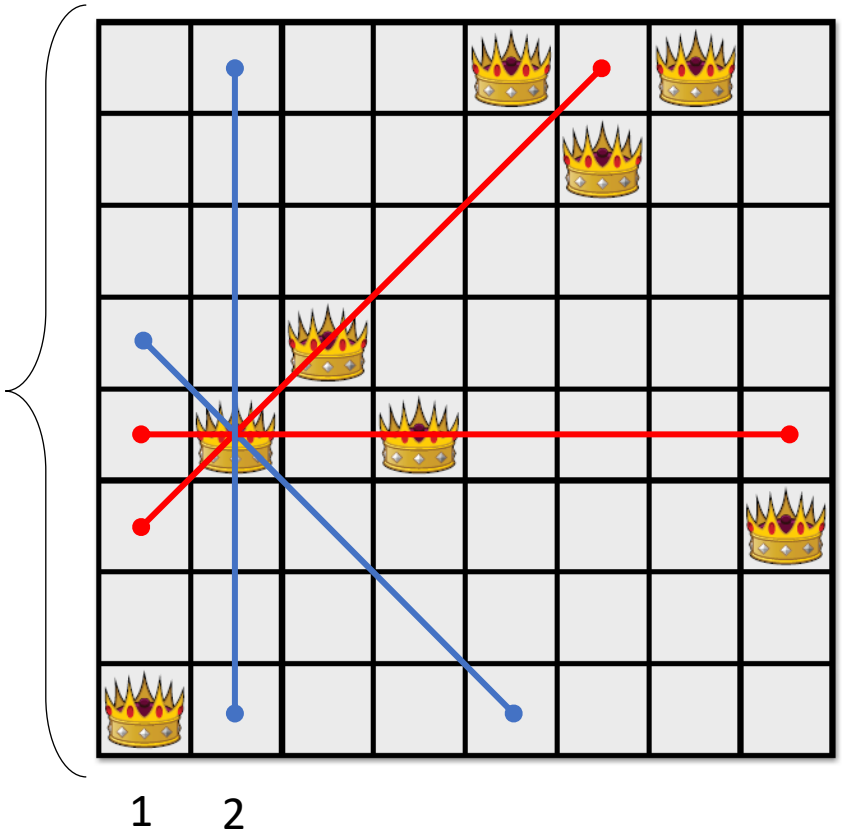
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



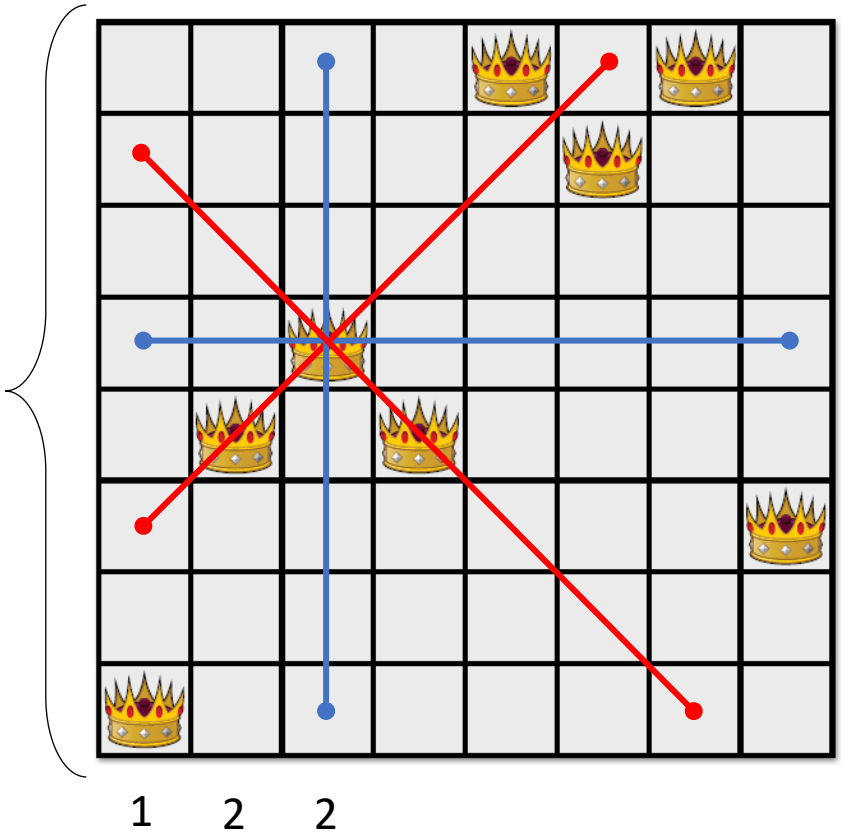
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



1 2 2

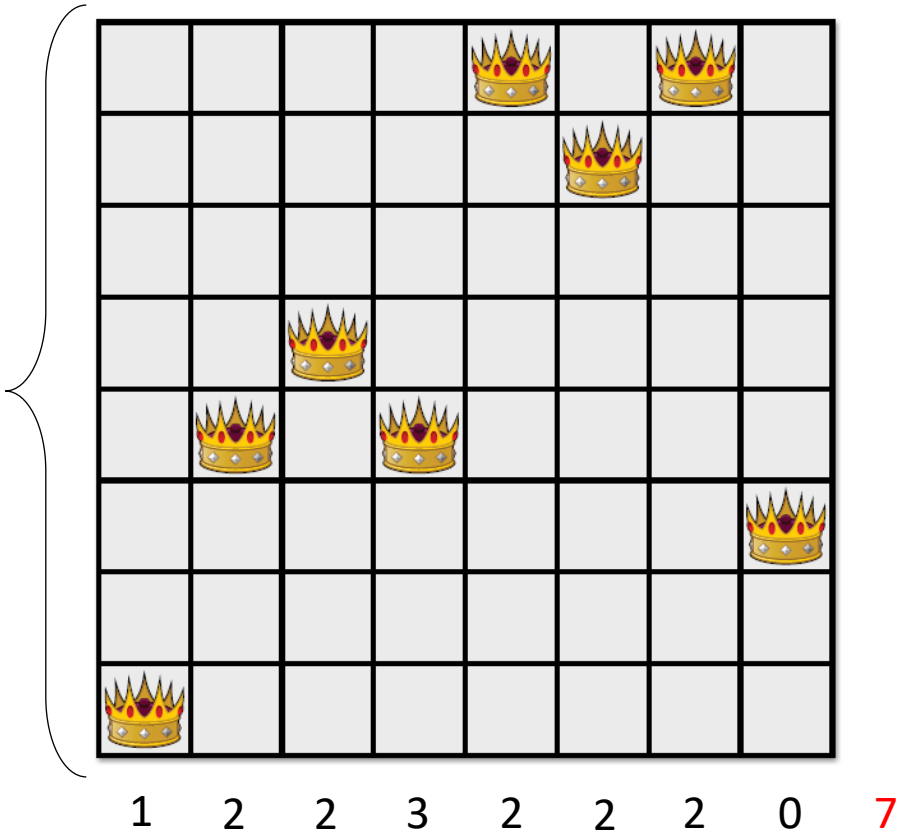
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



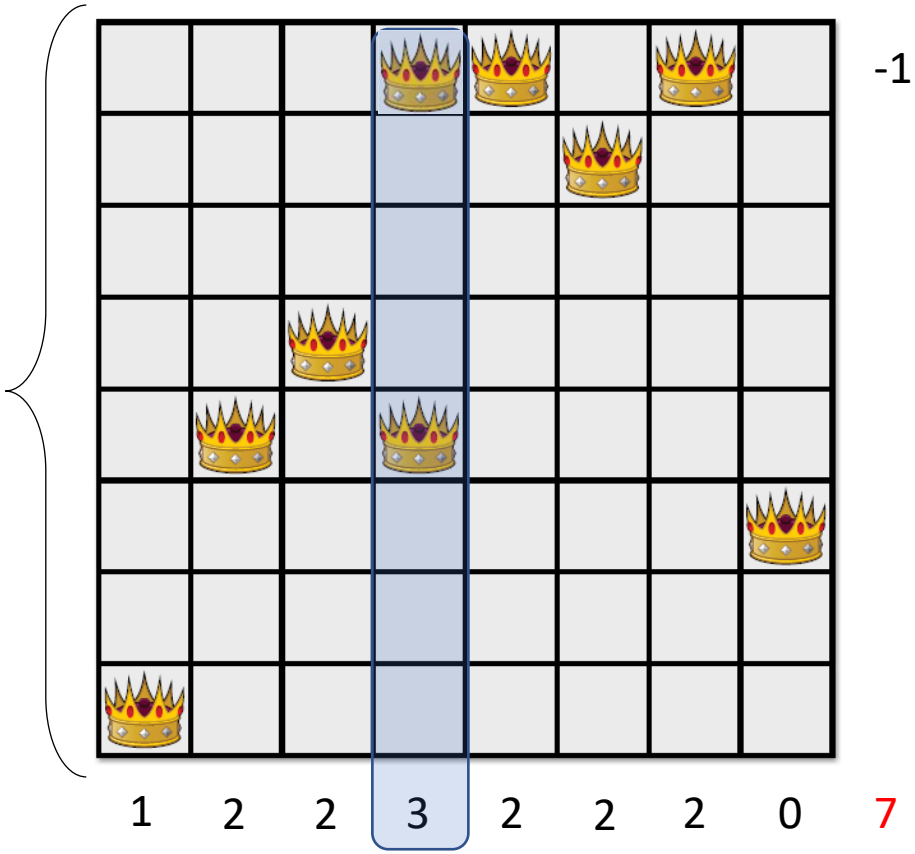
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



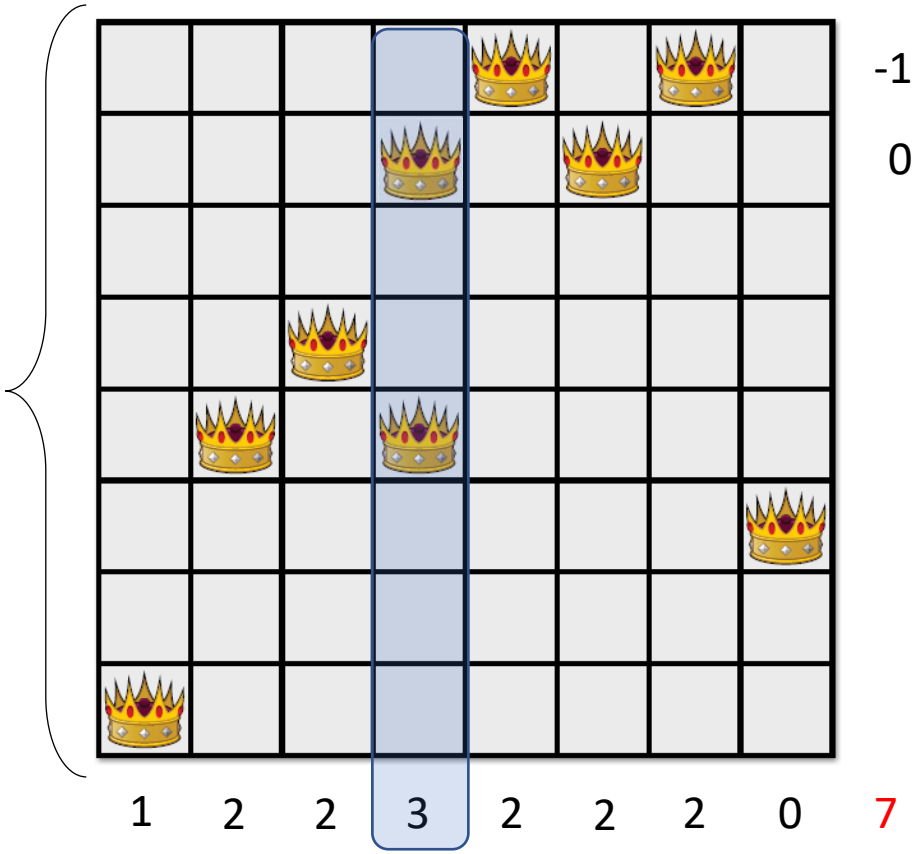
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



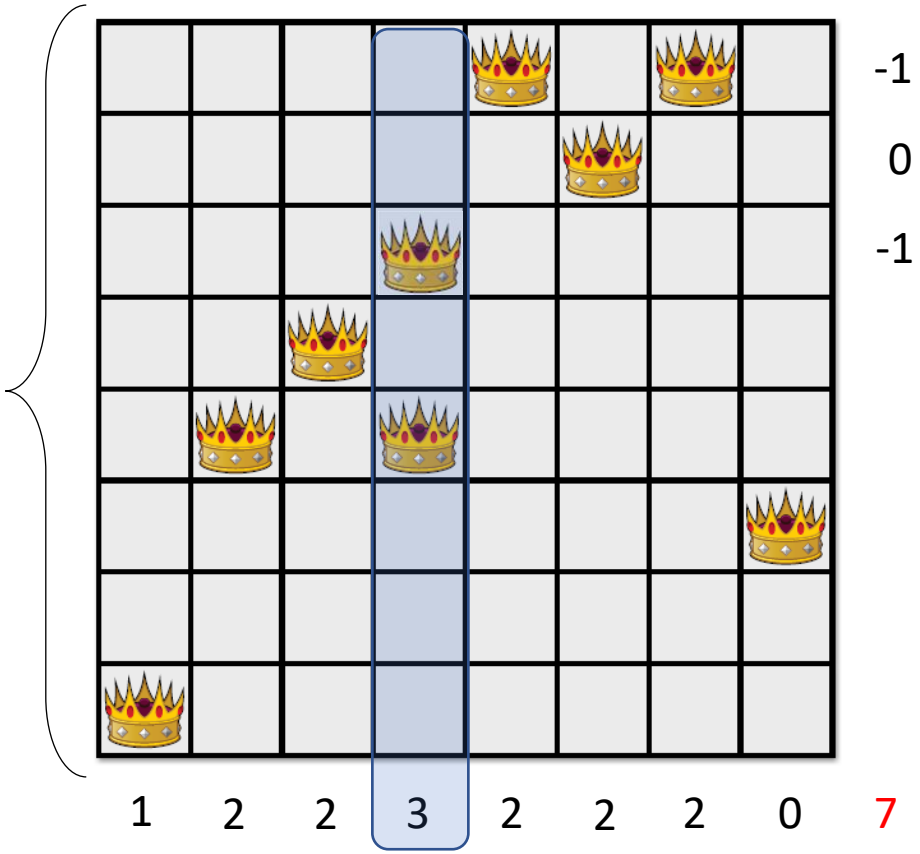
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



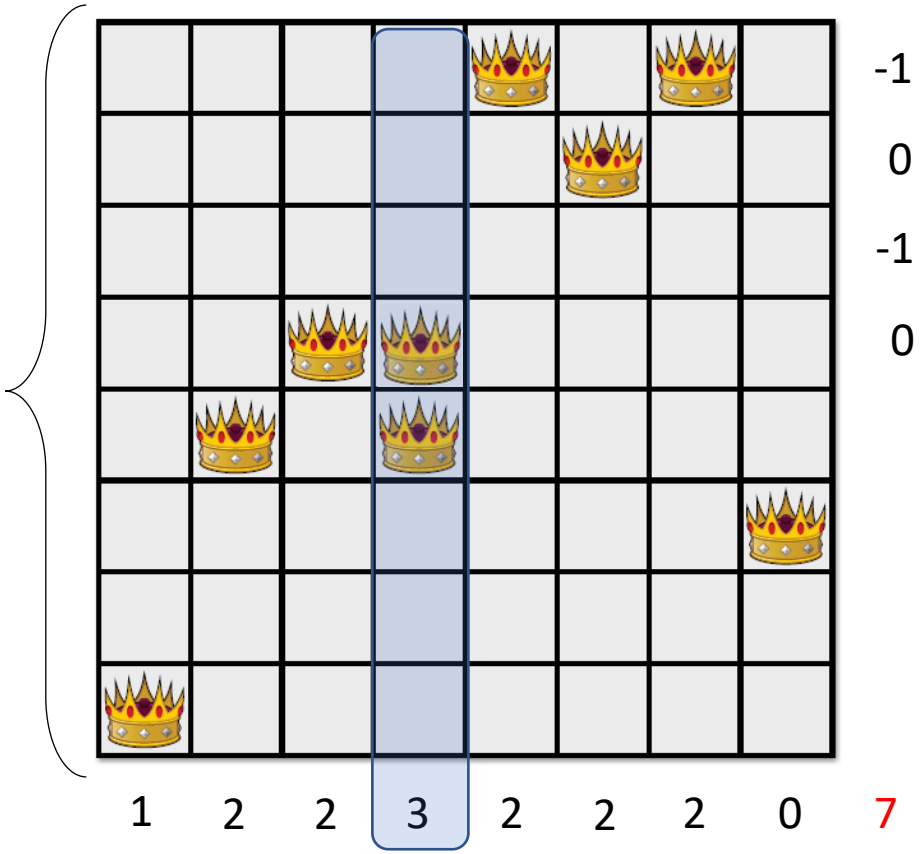
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



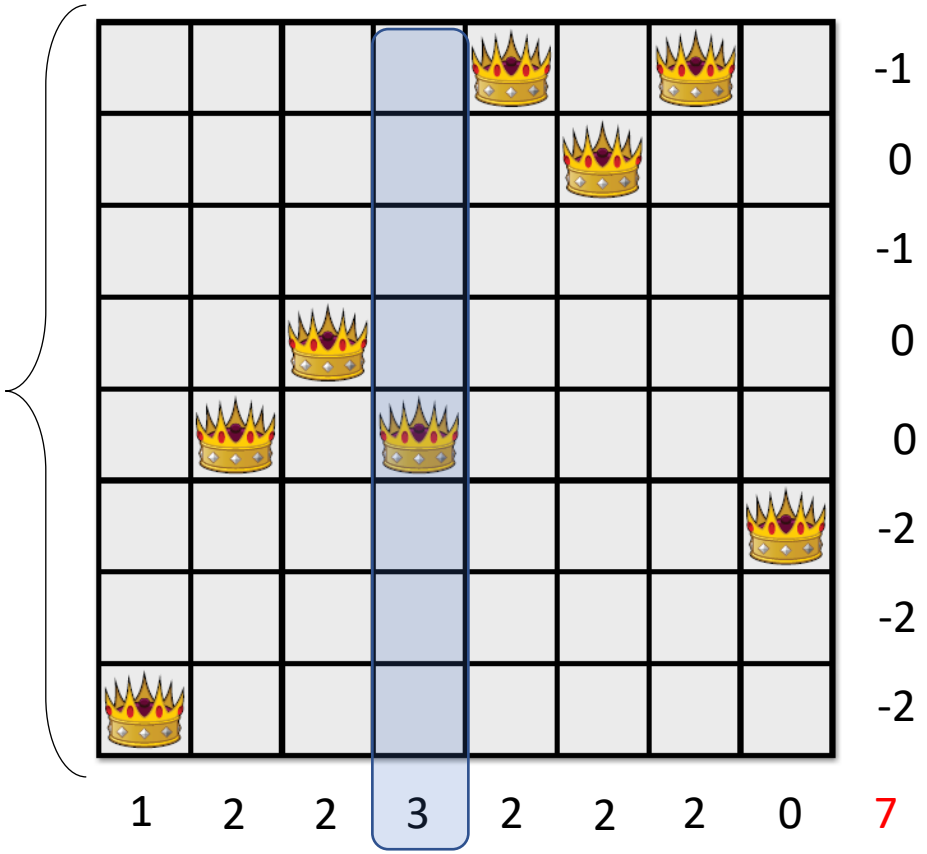
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



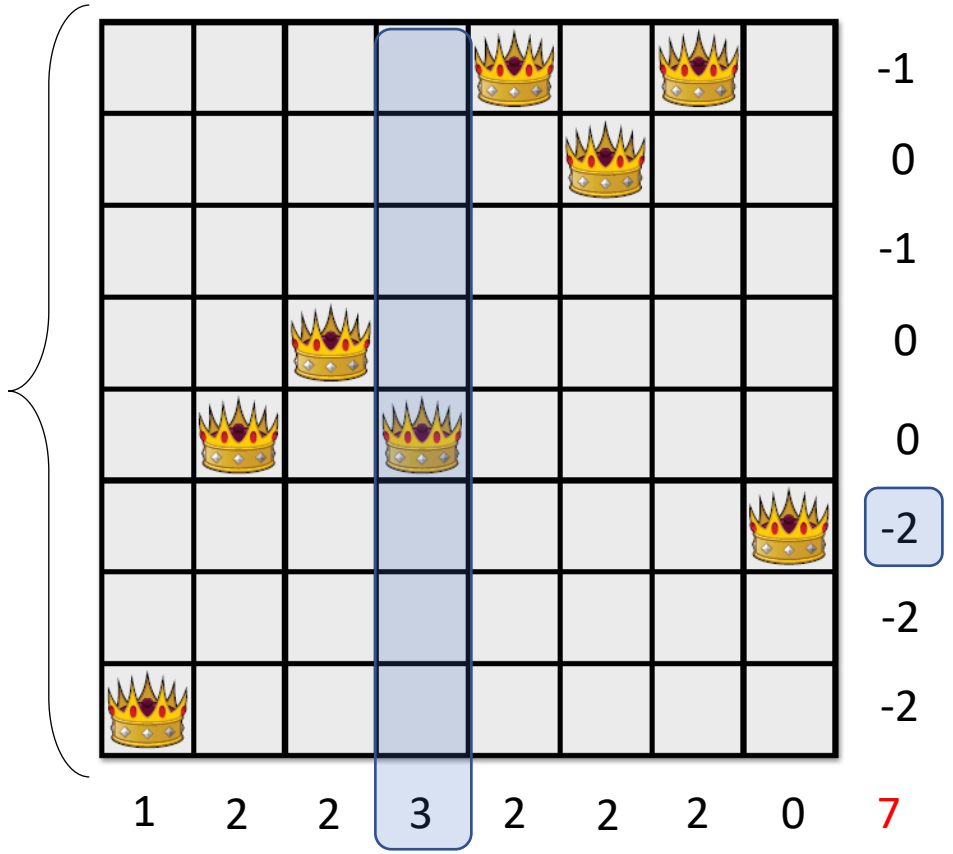
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



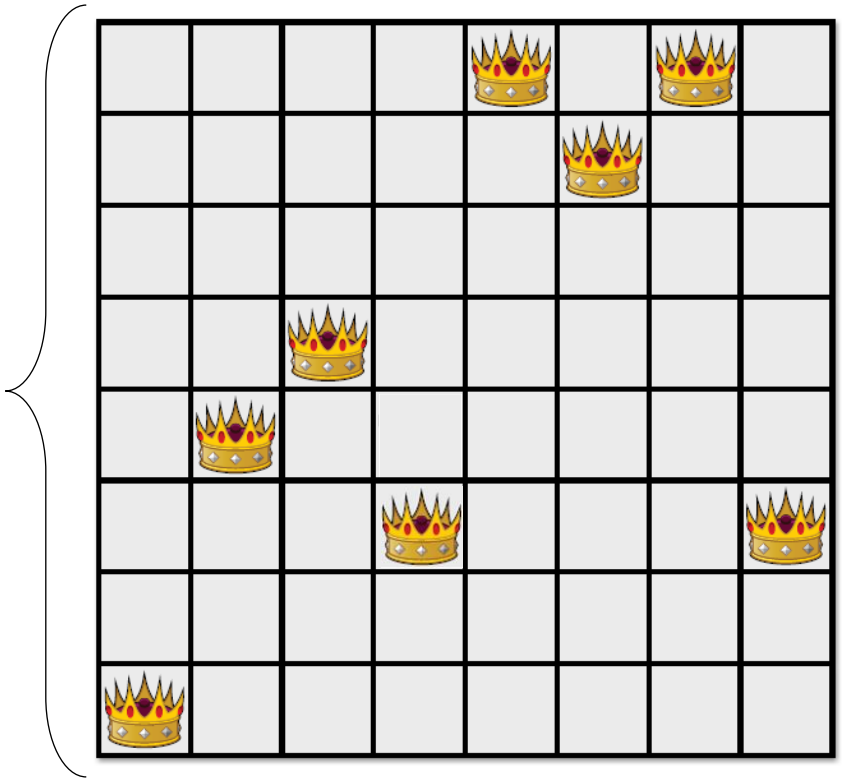
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



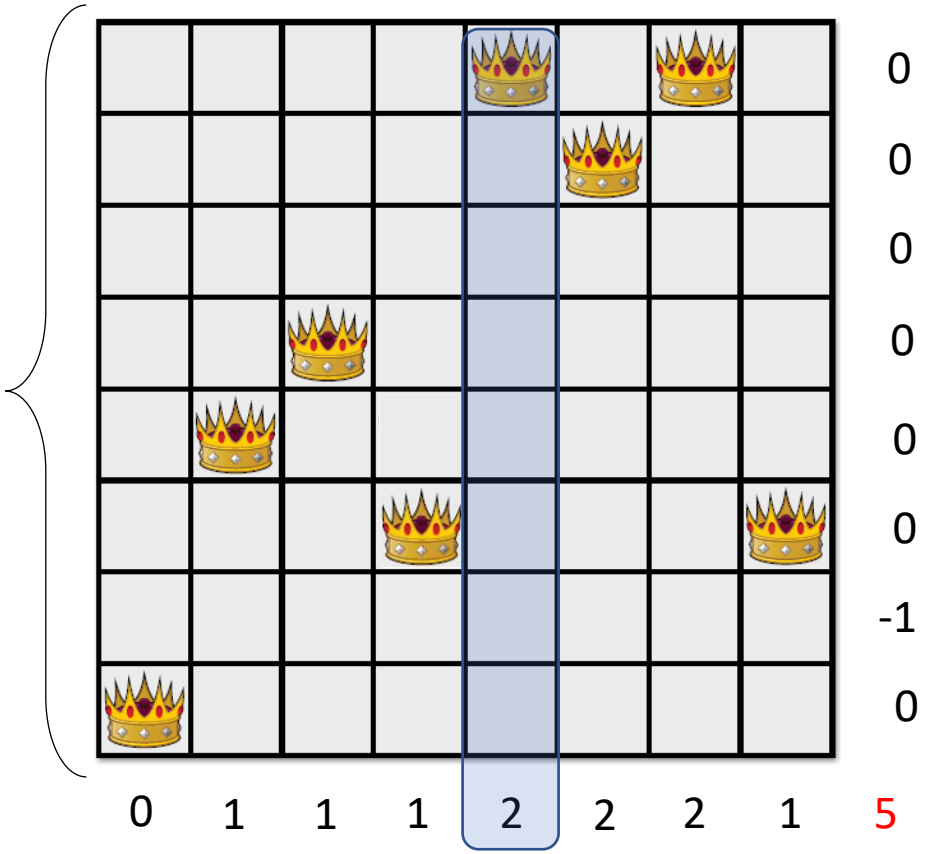
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



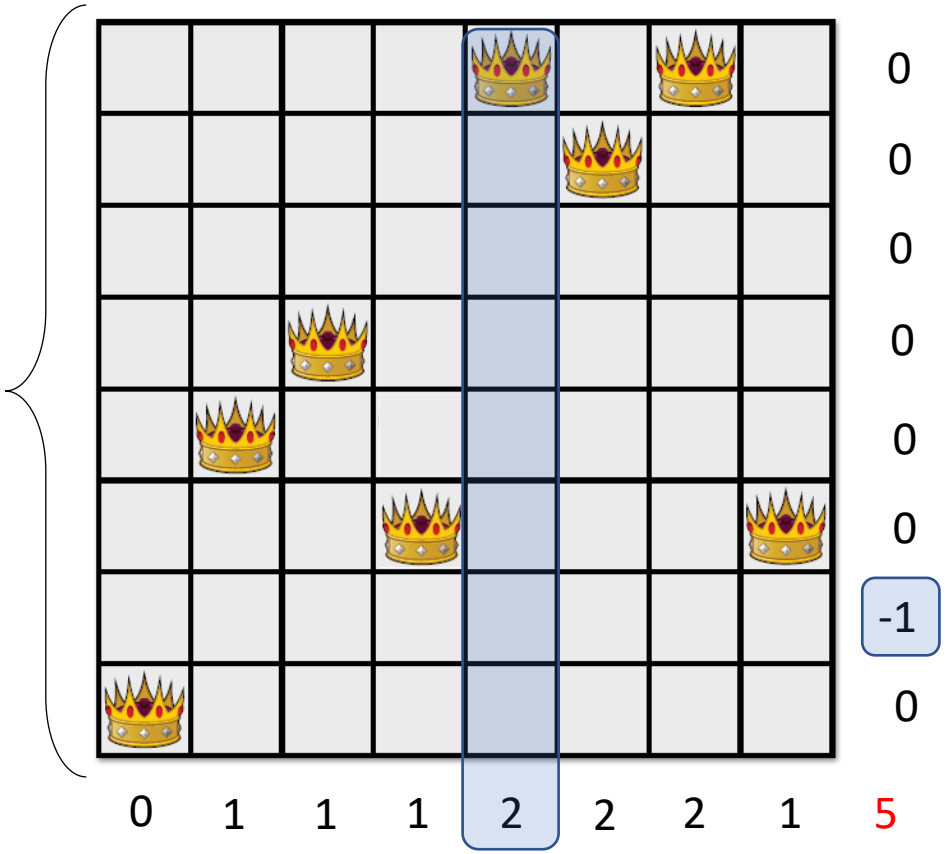
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



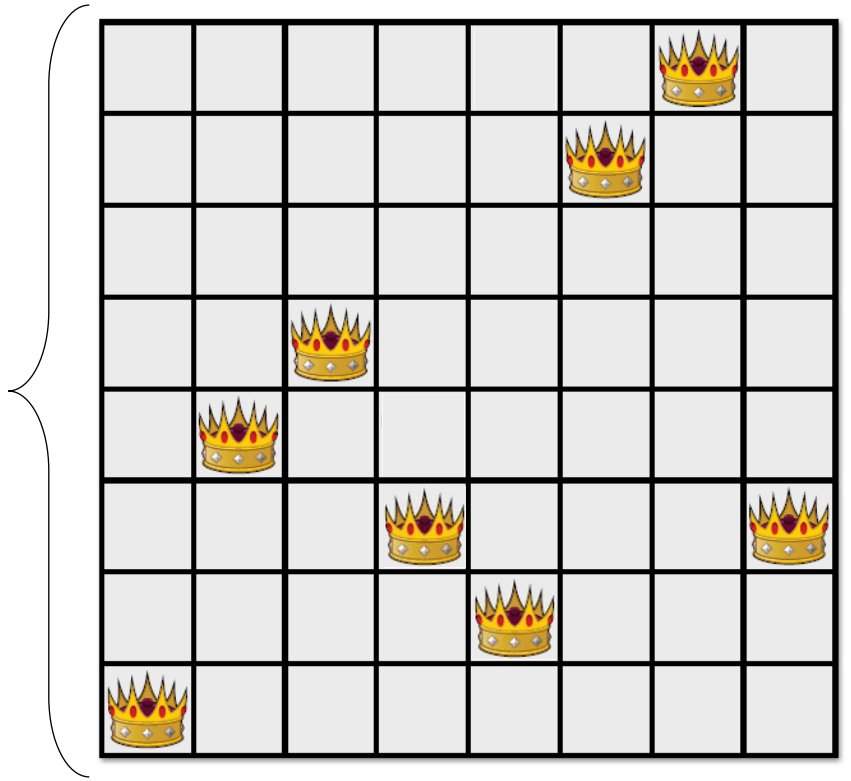
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



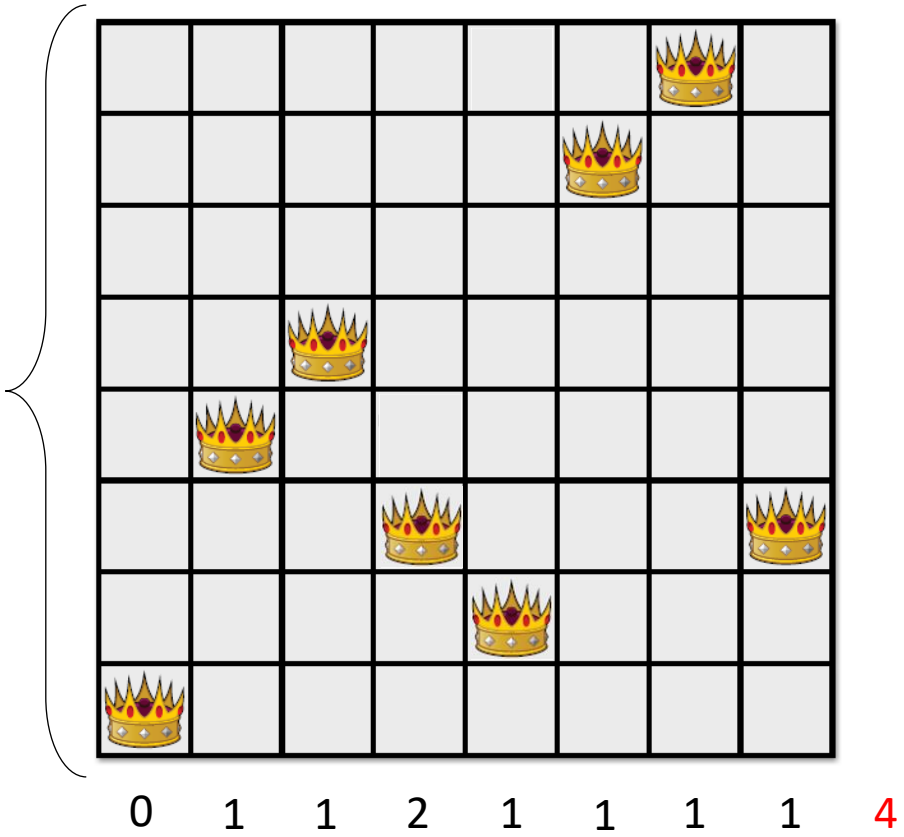
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



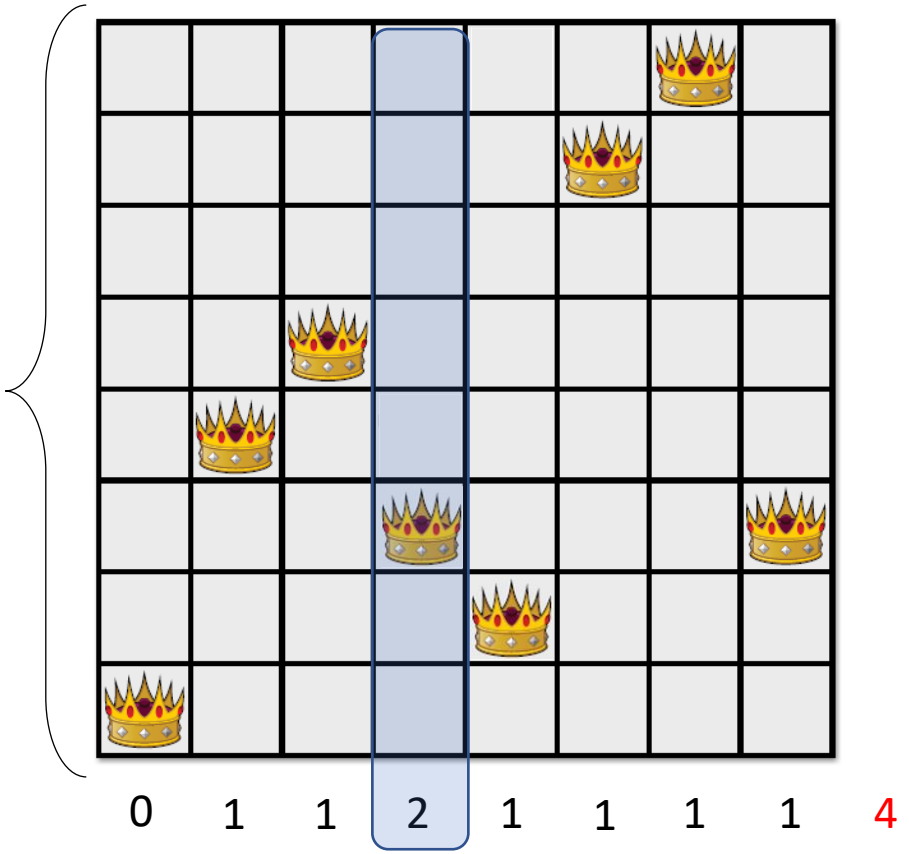
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



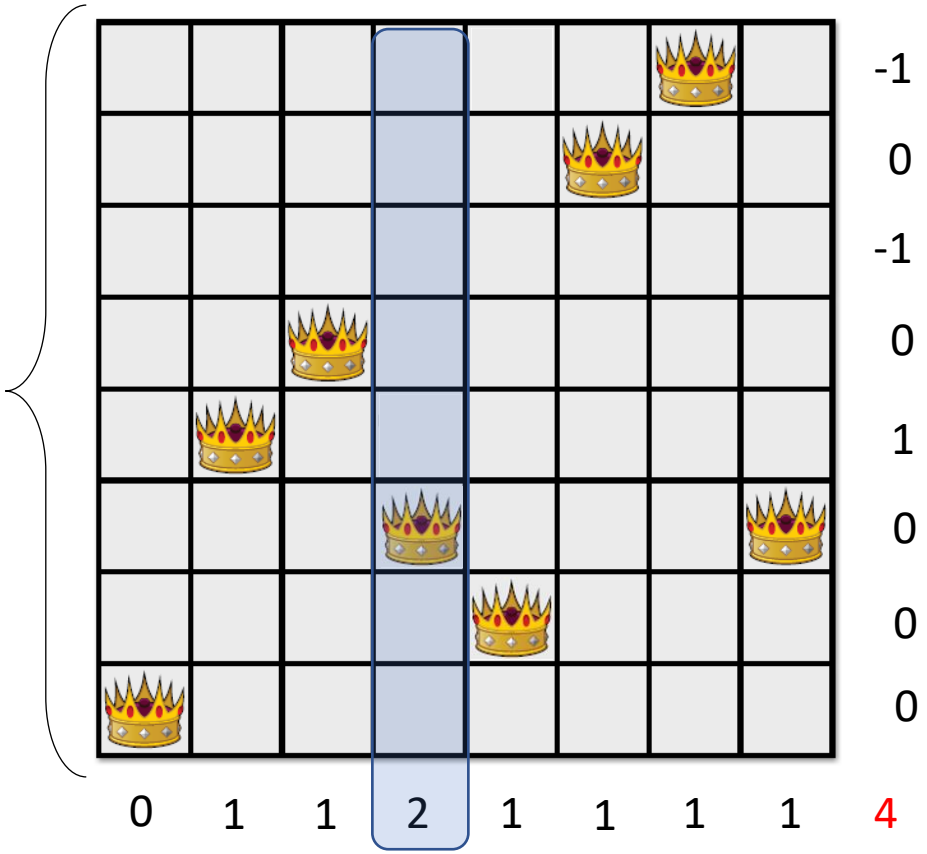
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



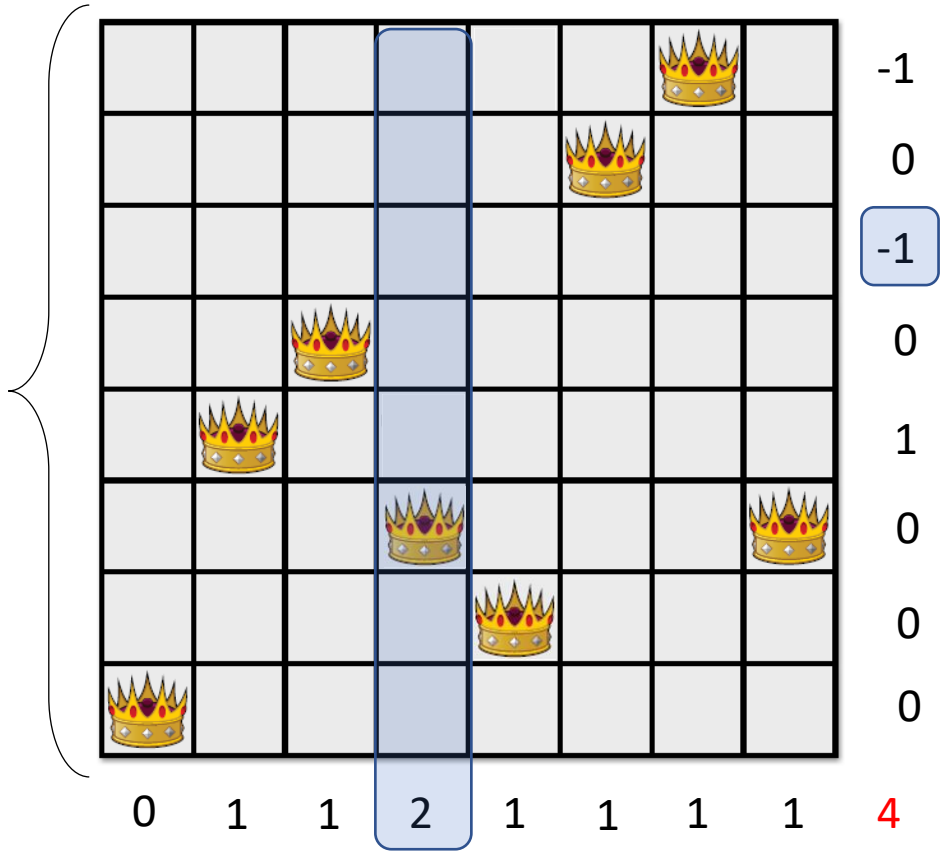
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas



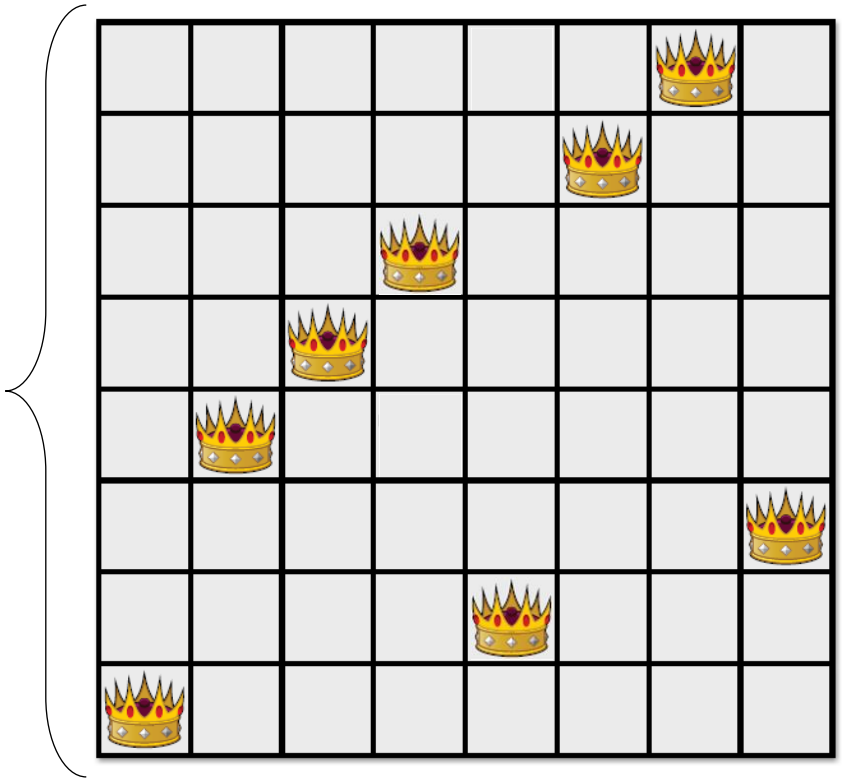
Algoritmos de Busca Local

Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade

Problema das n-rainhas

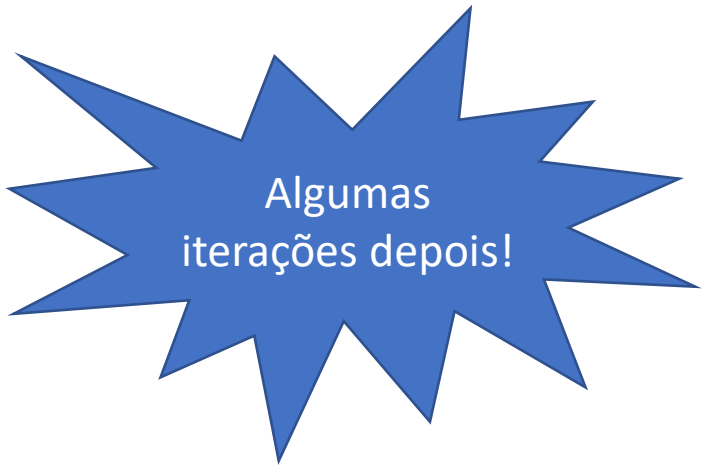


Algoritmos de Busca Local

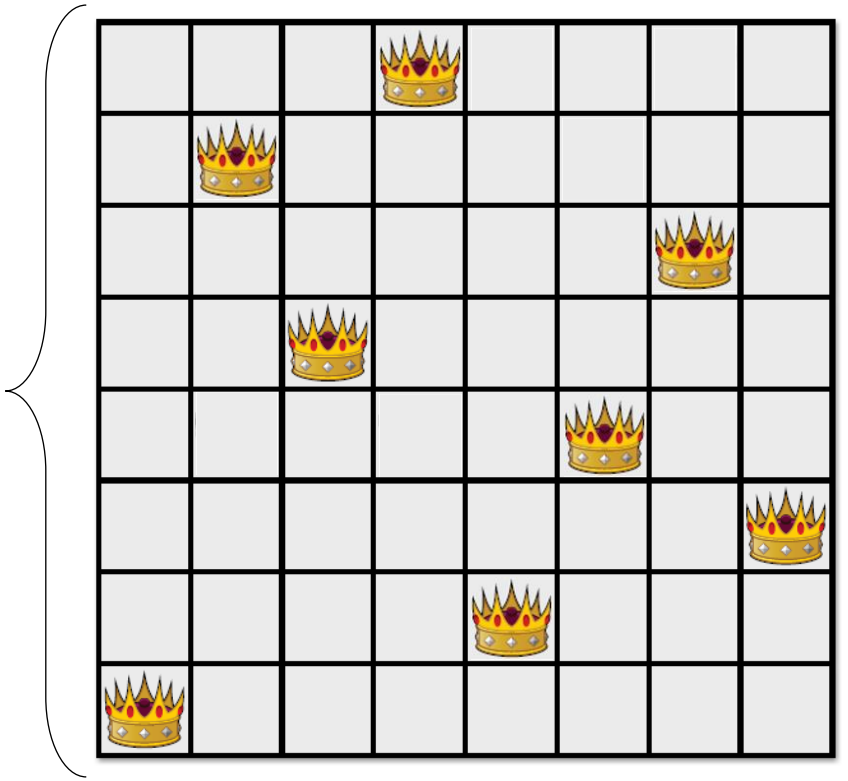
Problema de Satisfação

Definição: Encontrar uma solução viável para o problema, uma vez que qualquer solução viável possuí o mesmo valor de objetivo.

1. Inicia a partir de uma solução inviável
2. Faz movimentos em direção a viabilidade



Problema das n-rainhas



Algoritmos de Busca Local

Problema de Otimização: Problema da Mochila

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) SWAP
      i. Mantenha-se Viável, ou seja, mantenha a capacidade da mochila sempre respeitada.
      ii. Troque itens que estão na mochila por itens que não estão.



Problema da Mochila 0-1

Algoritmos de Busca Local

Problema de Otimização: Problema da Mochila

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) SWAP
      i. Mantenha-se Viável, ou seja, mantenha a capacidade da mochila sempre respeitada.
      ii. Troque itens que estão na mochila por itens que não estão.

Dentro da Mochila
(Cap Total 12)

ID:	1	2	4
Peso:	1	6	2,8

Fora da Mochila

ID:	3	5	
Peso:	5,1	4	



Problema da Mochila 0-1

Algoritmos de Busca Local

Problema de Otimização: Problema da Mochila

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) SWAP
      i. Mantenha-se Viável, ou seja, mantenha a capacidade da mochila sempre respeitada.
      ii. Troque itens que estão na mochila por itens que não estão.



Problema da Mochila 0-1

Dentro da Mochila (Cap Total 12)

ID:	1	2	4
Peso:	1	6	2,8

Fora da Mochila

ID:	3	5	
Peso:	5,1	4	

Movimento

Dentro da Mochila (Cap Total 12)

ID:	1	2	5
Peso:	1	6	4

Fora da Mochila

ID:	3	4	
Peso:	5,1	2,8	

Algoritmos de Busca Local

Problema de Otimização: Problema da Mochila

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) SWAP
      i. Mantenha-se Viável, ou seja, mantenha a capacidade da mochila sempre respeitada.
      ii. Troque itens que estão na mochila por itens que não estão.

Usar a Função Objetivo como guia para aceitar ou não uma troca!!!



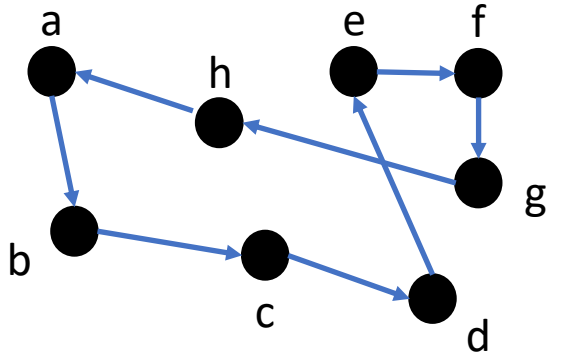
Problema da Mochila 0-1

Algoritmos de Busca Local

Problema de Otimização: Problema do Caixeiro Viajante Simétrico

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) 2-OPT
      i. Mantenha-se viável, isto é, manter o Ciclo Hamiltoniano
      ii. Selecione duas arestas e troque por duas outras arestas.



a → b → c → d → e → f → g → h → a

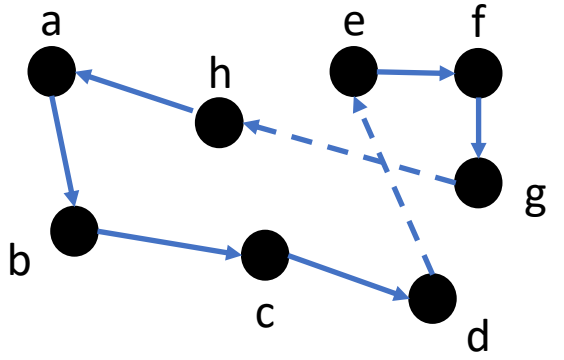
Uma solução do Problema do Caixeiro Viajante Simétrico

Algoritmos de Busca Local

Problema de Otimização: Problema do Caixeiro Viajante Simétrico

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) 2-OPT
      i. Mantenha-se viável, isto é, manter o Ciclo Hamiltoniano
      ii. Selecione duas arestas e troque por duas outras arestas.



a → b → c → d ⇸ e → f → g ⇸ h → a

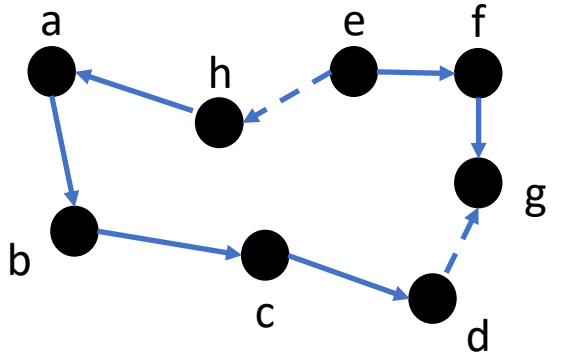
Uma solução do Problema do Caixeiro Viajante Simétrico

Algoritmos de Busca Local

Problema de Otimização: Problema do Caixeiro Viajante Simétrico

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) 2-OPT
      i. Mantenha-se viável, isto é, manter o Ciclo Hamiltoniano
      ii. Selecione duas arestas e troque por duas outras arestas.



a → b → c → d ⇸ g ← f ← e ⇸ h → a

Uma solução do Problema do Caixeiro Viajante Simétrico

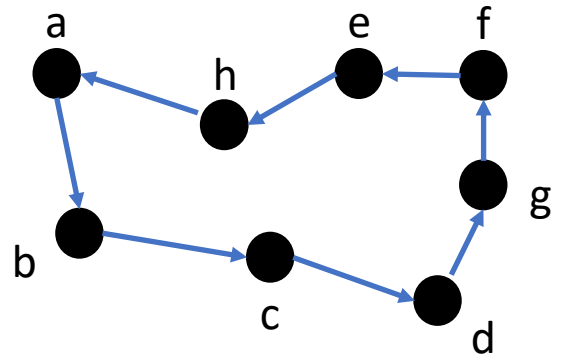
Algoritmos de Busca Local

Problema de Otimização: Problema do Caixeiro Viajante Simétrico

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local
   a) 2-OPT
      i. Mantenha-se viável, isto é, manter o Ciclo Hamiltoniano
      ii. Selecione duas arestas e troque por duas outras arestas.

   b) 3-OPT, 4-OPT, 5-OPT, K-OPT....



a → b → c → d → g → f → e → h → a

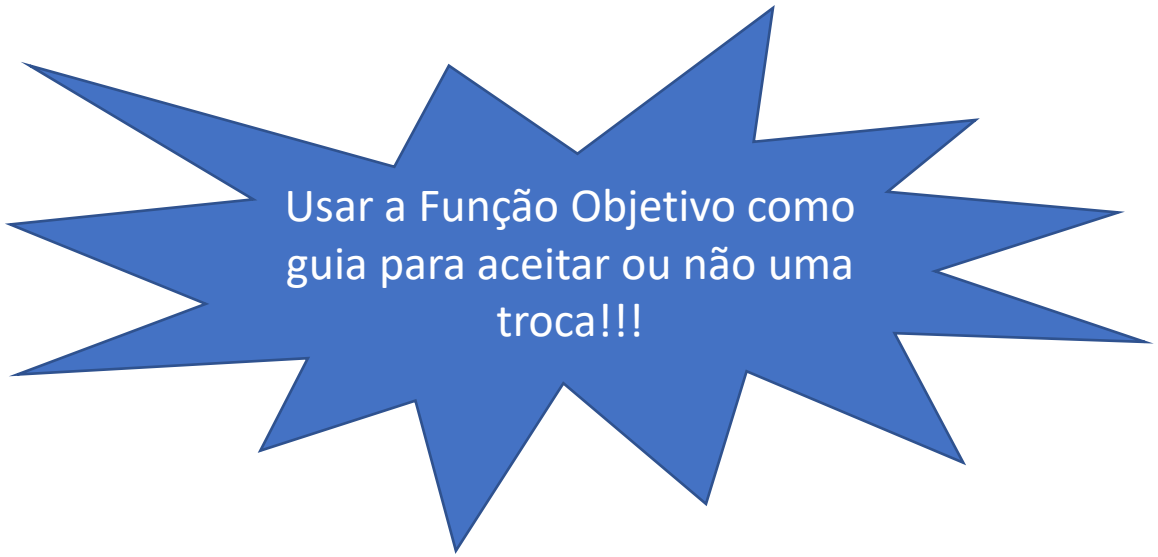
Uma solução do Problema do Caixeiro Viajante Simétrico

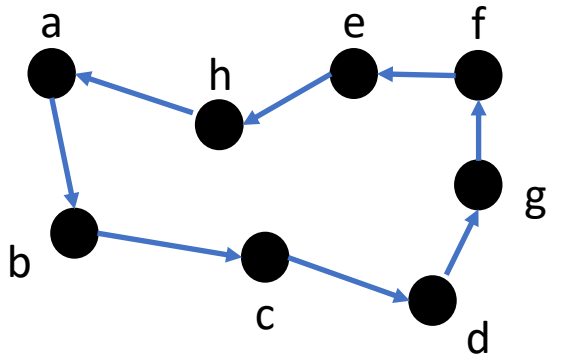
Algoritmos de Busca Local

Problema de Otimização: Problema do Caixeiro Viajante Simétrico

Definição: Encontrar uma solução ótima local para o problema, partindo de uma solução inicial

1. Inicia a partir de uma solução viável subótima
2. Faz movimentos em direção a uma solução ótima local





a → b → c → d → g → f → e → h → a

Uma solução do Problema do Caixeiro Viajante Simétrico

Agenda

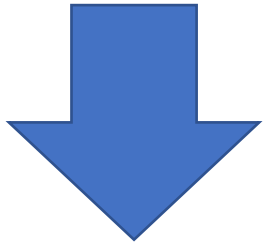
- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Introdução a Meta-heurísticas

Definições do National Institute of Standards and Technology

A high-level algorithmic framework or approach that can be specialized to solve optimization problems.

A high-level strategy that guides other heuristics in a search for feasible solutions.



Uma combinação de diferentes estratégias para percorrer o espaço de busca com mecanismos para escapar de ótimos locais de baixa qualidade. Tais métodos também não certificam otimalidade.

Introdução a Meta-heurísticas

Componentes básicos

1. Construções gulosas
2. Aleatorização
3. Busca local
4. Multiplicidade de vizinhanças
5. Memória
6. Intensificação
7. Diversificação

Introdução a Meta-heurísticas

Propriedades Desejáveis

1. Simplicidade: baseadas em princípios simples e claros.

2. Coerência: todos os passos da heurística para problemas particulares devem seguir naturalmente os princípios da meta-heurística.

3. Eficiência: devem fornecer *boas* soluções (ótimas ou quase ótimas) para todas ou quase todas as instâncias.

4. Efetividade: baixo tempo computacional.

5. Robustez: performance consistente sobre uma variedade de instâncias.

6. Portabilidade: poucos parâmetros.

7. Inovação: capacidade de “solucionar” novos tipos de aplicações.

Introdução a Meta-heurísticas

Classificações

- Há várias maneiras de se classificar e descrever as meta-heurísticas. Dependendo de cada uma das características escolhidas para diferenciá-las, podem-se criar várias classificações. A seguir as duas mais frequentemente encontradas na literatura:

- Inspiradas na natureza *versus* não inspiradas na natureza (Iterativas, de trajetória, baseadas em vizinhança:

    - A mais intuitiva das classificações, pois se refere à origem, à motivação para a criação daquela meta-heurística;

    - Apesar de intuitiva, esta classificação não é muito útil, porque os algoritmos híbridos, por exemplo, não se encaixam em nenhuma das duas classificações.

Introdução a Meta-heurísticas

Classificações

- Característica utilizada para a classificação: número de soluções utilizadas em cada iteração:
  - Uma população de soluções (meta-heurísticas populacionais);
  - Uma única solução (meta-heurísticas iterativas ou pontuais).

Os algoritmos pontuais geram uma trajetória no espaço de busca, enquanto os algoritmos populacionais descrevem a evolução de um conjunto de pontos do espaço de busca.

Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Greedy Randomized Adaptive Search Procedure (GRASP)

Introdução

“Probabilistic Heuristic”

Feo, T.A., Resende, M.G.C.: *A probabilistic heuristic for a computationally difficult set covering problem*. Op. Res. Lett. **8**, 67–71 (1989)

Greedy Randomized Adaptive Search Procedure (GRASP)

Introdução

"Probabilistic Heuristic"

Feo, T.A., Resende, M.G.C.: *A probabilistic heuristic for a computationally difficult set covering problem*. Op. Res. Lett. **8**, 67–71 (1989)



"GRASP"

Feo, T.A., Resende, M.G.C.: Greedy Randomized Adaptive Search Procedures. Journal of Global Optimization. 6(2), 109–133 (1995)

Greedy Randomized Adaptive Search Procedure (GRASP)

Pseudo-código

Feo, T.A., Resende, M.G.C.: Greedy Randomized Adaptive Search Procedures. Journal of Global Optimization. 6(2), 109–133 (1995)

```
function GRASP(α, Critério_de_Parada)
    s* ← ∅;
    repeat
        s ← Construção(α);
        s ← Busca_Local(s);
        if (f(s) < f(s*)) then
            s* ← s;
        end-if
    until Critério_de_Parada
    return s*;
end
```

Greedy Randomized Adaptive Search Procedure (GRASP)

Pseudo-código

Feo, T.A., Resende, M.G.C.: Greedy Randomized Adaptive Search Procedures. Journal of Global Optimization. 6(2), 109–133 (1995)

```
function GRASP(α, Critério_de_Parada)
    s* ← ∅;
    repeat
        s ← Construção(α);
        s ← Busca_Local(s);
        if (f(s) < f(s*)) then
            s* ← s;
        end-if
    until Critério_de_Parada
    return s*;
end
```

Laço de repetição até um determinado Critério de Parada

Construtivo que utiliza Lista de Candidatos Restrita (α)

Greedy Randomized Adaptive Search Procedure (GRASP)

Hibridização – *High Performance Computing* (HPC)

```
function GRASP(α, Critério_de_Parada)
    s* ← ∅;
    repeat
        s ← Construção(α);
        s ← Busca_Local(s);
        if(f(s) < f(s*))then
            s* ← s;
        end-if
    until Critério_de_Parada
    return s*;
end
```



Cada iteração do laço de repetição é resolvida paralelamente em uma *thread* diferente.

Greedy Randomized Adaptive Search Procedure (GRASP)

Hibridização – *High Performance Computing* (HPC)

```
function GRASP(α, Critério_de_Parada)
    s* ← ∅;
    repeat
        s ← Construção(α);
        s ← Busca_Local(s);
        if (f(s) < f(s*)) then
            s* ← s;
        end-if
    until Critério_de_Parada
    return s*;
end
```



Cada iteração do laço de repetição é resolvida paralelamente em uma *thread* diferente.

Cuidado na atualização concorrente de s^* !!!!

Greedy Randomized Adaptive Search Procedure (GRASP)

Hibridização – Mineração de Dados

function GRASP-DM(α, $Critério_de_Parada$)
    $s^* \leftarrow \emptyset$;
    Candidatos_Elite ← ∅;
    repeat
        $s \leftarrow Construção(\alpha)$;
        $s \leftarrow Busca_Local(s)$;
        Candidatos_Elite.add(s);
        $if\big(f(s) < f(s^*)\big) then$
            $s^* \leftarrow s$;
        end-if
    until $Critério_de_Parada/2$;
    $Padroes_Minerados \leftarrow Minerador\ (Candidatos_Elite)$;
    repeat
        $s \leftarrow Construção(\alpha, Padroes_Minerados)$;
        $s \leftarrow Busca_Local(s)$;
        $if\big(f(s) < f(s^*)\big) then$
            $s^* \leftarrow s$;
        end-if
    until $Critério_de_Parada/2$
    return s^*;
end

Greedy Randomized Adaptive Search Procedure (GRASP)

Hibridização – Mineração de Dados

function GRASP-DM(α, $Critério_de_Parada$)

$s^* \leftarrow \emptyset$;

Candidatos_Elite $\leftarrow \emptyset$;

repeat

$s \leftarrow Construção(\alpha)$;

$s \leftarrow Busca_Local(s)$;

Candidatos_Elite.add(s);

$if\left(f(s) < f(s^*)\right) then$

$s^* \leftarrow s$;

end-if

until $Critério_de_Parada/2$;

$Padroes_Minerados \leftarrow Minerador\ (Candidatos_Elite)$;

Frequent Itemset Mining (FIM)

"Blocos" GRASP

repeat

$s \leftarrow Construção(\alpha, Padroes_Minerados)$;

$s \leftarrow Busca_Local(s)$;

$if\left(f(s) < f(s^*)\right) then$

$s^* \leftarrow s$;

end-if

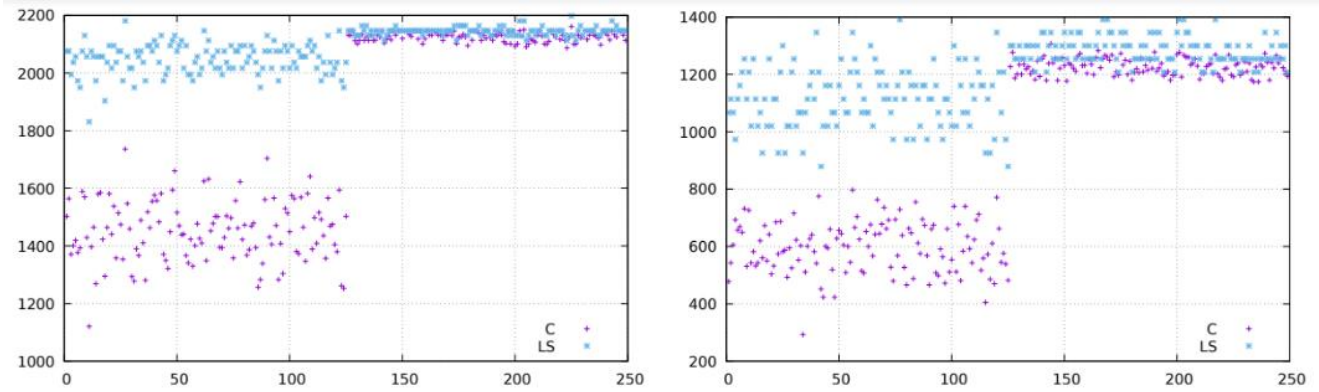
until $Critério_de_Parada/2$

return s^*;

end

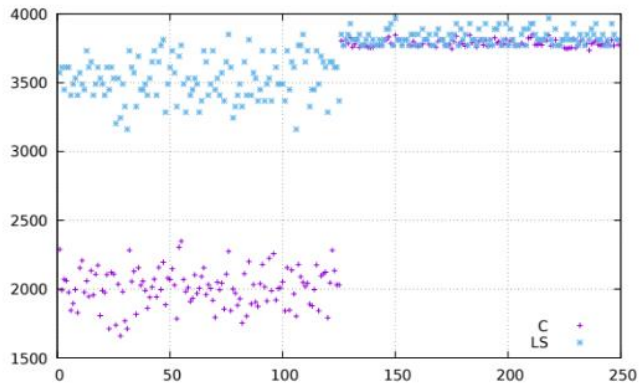
Greedy Randomized Adaptive Search Procedure (GRASP)

Hibridização – Mineração de Dados



(a) Instancia com 100 itens

(b) Instancia com 260 itens



(c) Instancia com 360 itens

Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Variable Neighborhood Search e *Iterated Local Search*

Introdução

"VNS"

Nenad Mladenović; Pierre Hansen (1997). "Variable neighborhood search". Computers and Operations Research. **24**(11): 1097–1100.

"ILS"

Lourenço, H.R. (1995). "Job-Shop Scheduling: computational study of local search and large-step optimization methods". European Journal of Operational Research. **83** (2): 347–364.

Lourenço, H.R.; Zwijnenburg M. (1996). Combining the large-step optimization with tabu-search: application to the job-shop scheduling problem. Meta-Heuristics: Theory and Applications. Kluwer Academic Publishers. Springer. pp. 219–236.

Lourenço, H.R.; Martin O.; Stützle T. (2003). "Iterated Local Search". Handbook of Metaheuristics. Kluwer Academic Publishers, International Series in Operations Research & Management Science. **57**: 321–353.

Variable Neighborhood Search e *Iterated Local Search*

Pseudo-código

```
function VNS(s, k_max, t_max)
    repeat
        k ← 1;
        repeat
            s' ← Shake(s, k);
            s'' ← LocalSearch(s');
            AcceptanceCriterion(s'', s*);
            s ← NeighbourhoodChange(s, s'', k);
        until k = k_max
        t ← CpuTime();
    until t > t_max
    return s;
end
```

```
function ILS()
        s ← GenerateInitialSolution( );
        {s*, s_b} ← LocalSearch(s);
        repeat
            s' ← Perturbation(s_b);
            s*' ← LocalSearch(s');
            s_b ← AcceptanceCriterion(s*', s*);
        until termination condition met
        return s*;
end
```

Variable Neighborhood Search e *Iterated Local Search*

Hibridização – *High Performance Computing* (HPC)

function VNS(s, k_{max}, t_{max})
repeat
$k \leftarrow 1$;
repeat
$s' \leftarrow Shake(s, k)$;
$s'' \leftarrow LocalSearch(s')$;
$AcceptanceCriterion(s'', s^*)$;
$s \leftarrow NeighbourhoodChange(s, s'', k)$;
until $k = k_{max}$
$t \leftarrow CpuTime()$;
until $t > t_{max}$
return s;
end

function ILS()
$s \leftarrow GenerateInitialSolution()$;
$\{s^*, s_b\} \leftarrow LocalSearch(s)$;
repeat
$s' \leftarrow Perturbation(s_b)$;
$s^{*\prime} \leftarrow LocalSearch(s')$;
$s_b \leftarrow AcceptanceCriterion(s_b, s^{*\prime}, s^*)$;
until termination condition met
return s^*;
end

Multi-Improvement Local Search
Parallel Local Search(N_k)

Variable Neighborhood Search e *Iterated Local Search*

Hibridização – Mineração de Dados

```
function VNS(s, k_max, t_max)
    repeat
        k ← 1;
        repeat
            s' ← Shake(s, k);
            s'' ← LocalSearch(s');
            AcceptanceCriterion(s'', s*);
            s ← NeighbourhoodChange(s, s'', k);
        until k = k_max
        t ← CpuTime();
    until t > t_max
    return s;
end
```

```
function ILS()
        s ← GenerateInitialSolution( );
        {s*, s_b} ← LocalSearch(s);
        repeat
            s' ← Perturbation(s_b);
            s*' ← LocalSearch(s');
            s_b ← AcceptanceCriterion(s_b, s*', s*);
        until termination condition met
        return s*;
end
```

Fixar os padrões minerados de modo que a perturbação não os modifique

Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Simulated Annealing

Introdução

“Simulated Annealing”

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. science, 220(4598), 671-680.

Simulated Annealing
Introdução

```
function SA(AS_max, T, t_0, α)
        {s*, s'} ← GenerateSolution();
        iter ← 0;
        t ← t_0;
        while (t > T) do
                while(iter < AS_max)
                        iter ← iter + 1;
                        s'' ← GetNeighbor(s');
                        Δ ← f(s')– f(s'');
                        if ( Δ ≥ 0)
                                s' ← s'';
                                if (f(s') < f(s*))
                                        s* ← s';
                        else
                                if (RAND(0,1) < e^{-Δ/t})
                                        s' ← s'';
                        end
                end
                t ← α × t;
                iter ← 0;
        end
        return s*;
end
```

Simulated Annealing

Hibridização – *High Performance Computing* (HPC)

```
function SA(AS_max, T, t_0, α)
        {s*, s'} ← GenerateSolution();
        iter ← 0;
        t ← t_0;
        while (t > T) do
            while(iter < AS_max)
                iter ← iter + 1;
                s'' ← GetNeighbor(s');
                Δ ← f(s')− f(s'');
                if ( Δ ≥ 0)
                    s' ← s'';
                    if (f(s') < f(s*))
                        s* ← s';
                else
                    if (RAND(0,1) < e^{(−Δ/t)})
                        s' ← s'';
                end
            end
            t ← α × t;
            iter ← 0;
        end
        return s*;
end
```

Múltiplos Instanciamentos do laço

"Multi-Walk"

Lista de Candidatos Restrita

Simulated Annealing

Hibridização – Mineração de Dados

```
function SA(AS_max, T, t_0, α)
        {s*, s'} ← GenerateSolution();
        iter ← 0;
        t ← t_0;
        while (t > T) do
                while(iter < AS_max)
                        iter ← iter + 1;
                        s'' ← GetNeighbor(s');
                        Δ ← f(s')− f(s'');
                        if ( Δ ≥ 0)
                                s' ← s'';
                                if (f(s') < f(s*))
                                        s* ← s';
                        else
                                if (RAND(0,1) < e^{-Δ/t})
                                        s' ← s'';
                        end
                end
                t ← α × t;
                iter ← 0;
        end
        return s*;
end
```

"Guided-Walk"

Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Genetic Algorithm
Introdução

“Genetic Algorithm”

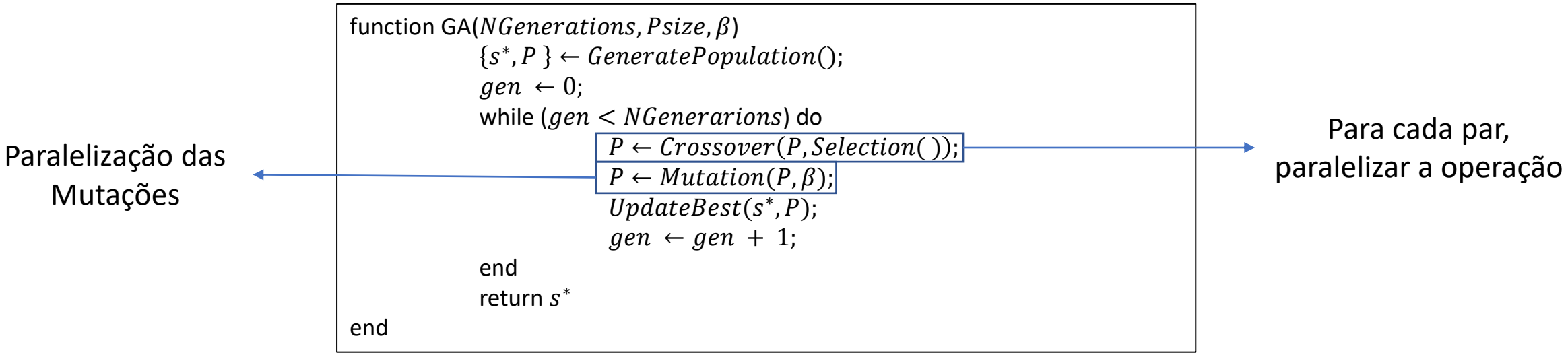
Holland, J.H.: Adaptation in Natural and Artificial Systems. University of Michigan Press, Ann Arbor, Michigan; re-issued by MIT Press (1992) (1975)

Genetic Algorithm
Introdução

```
function GA(NGenerations, Psize, β)
        {s*, P } ← GeneratePopulation();
        gen ← 0;
        while (gen < NGenerarions) do
                P ← Crossover(P, Selection( ));
                P ← Mutation(P, β);
                UpdateBest(s*, P);
                gen ← gen + 1;
        end
        return s*
end
```

Genetic Algorithm

Hibridização – *High Performance Computing* (HPC)



Genetic Algorithm

Hibridização – Mineração de Dados

Fixar elementos pertencentes a Padrões Frequentes Minerados

```
function GA(NGenerations, Psize, β)
        {s*, P } ← GeneratePopulation();
        gen ← 0;
        while (gen < NGenerarions) do
                P ← Crossover(P, Selection( ));
                P ← Mutation(P, β);
                UpdateBest(s*, P);
                gen ← gen + 1;
        end
        return s*
end
```

Priorizar herança de “alelos” pertencentes a Padrões Frequentes Minerados

Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Tabu Search

Introdução

“Local Search based Search”

Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. *Computers & operations research*, *13*(5), 533-549.

“Tabu Search”

Glover, F. (1989). Tabu search—part I. *ORSA Journal on computing*, *1*(3), 190-206.
Glover, F. (1990). Tabu search—part II. *ORSA Journal on computing*, *2*(1), 4-32.

Tabu Search
Introdução

```
function Tabu(s_0, StoppingCriteria(), maxTabuSize)
        {s*, s̄} ← s_0 ;
        tabuList ← [ ];
        tabuList.push(s_0);
        while (! StoppingCriteria()) do
                sNeighborhood ← getNeighbors(s̄);
                for (sCandidate ∈ sNeighborhood)
                        if(! In_tabuList(sCandidate) && f(sCandidate) > f(s̄))
                                s̄ ← sCandidate;
                        end
                end
                UpdateBest(s*, s̄);
                tabuList.push(s̄);
                if(tabuList.size( ) > maxTabuSize)
                        tabuList.removeFirst( );
                end
        end
        return s*
end
```

Tabu Search

Hibridização – Mineração de Dados

Priorizar não só o f(.) mas também padrões frequentes

```
function Tabu(s_0, StoppingCriteria(), maxTabuSize)
        {s*, s̄} ← s_0 ;
        tabuList ← [ ];
        tabuList.push(s_0);
        while (! StoppingCriteria()) do
                sNeighborhood ← getNeighbors(s̄);
                for (sCandidate ∈ sNeighborhood)
                        if(! In_tabuList(sCandidate) && f(sCandidate) > f(s̄))
                                s̄ ← sCandidate;
                        end
                end
                UpdateBest(s*, s̄);
                tabuList.push(s̄);
                if(tabuList.size( ) > maxTabuSize)
                        tabuList.removeFirst( );
                end
        end
        return s*
end
```

Reduzir o tempo na lista tabu em função dos padrões frequentes

Tabu Search

Hibridização – *High Performance Computing* (HPC)

```
function Tabu(s_0, StoppingCriteria(), maxTabuSize)
        {s*, s̄} ← s_0 ;
        tabuList ← [ ];
        tabuList.push(s_0);
        while (! StoppingCriteria()) do
                sNeighborhood ← getNeighbors(s̄);
                for (sCandidate ∈ sNeighborhood)
                        if(! In_tabuList(sCandidate) && f(sCandidate) > f(s̄))
                                s̄ ← sCandidate;
                        end
                end
                UpdateBest(s*, s̄);
                tabuList.push(s̄);
                if(tabuList.size( ) > maxTabuSize)
                        tabuList.removeFirst( );
                end
        end
        return s*
end
```

Gerar os vizinhos em paralelo

Determinar movimento não-Tabu

Agenda

- Introdução
- Algoritmos Construtivos
- Algoritmos de Busca Local
- Introdução a Meta-heurística
- *Greedy Randomized Adaptive Search Procedure*
- *Variable Neighborhood Search* e *Iterated Local Search*
- *Simulated Annealing*
- *Genetic Algorithm*
- *Tabu Search*
- *Heurísticas Primais usando MIP*

Heurísticas Primais usando MIP

Por simplicidade, a maioria das heurísticas é descrita para o PI: $\max\{cx : x \in X \subset Z_+^n\}$, mas são facilmente generalizadas para PIs Misto: $\max\{cx : x \in X \subset Z_+^n \times R^p\}$.

- Heurística Construtivas
- Heurísticas de Aprimoramento

Heurísticas Primais usando MIP

Heurísticas Construtívas

1. Arredondamento: Selecione uma variável x_j que deve ser inteira e arredonde para o inteiro mais próximo, denotado $[x_j]$;
2. Deslocamento: Desloque x_j para $x_j \pm \alpha$ se isso diminuir a inviabilidade.
3. Fixar: Defina x_j para algum valor $x_j' \in Z$ (usando pré-processamento ou programação por restrições para apertar os limites devido à fixação).

Se a repetição desses passos não levar a uma solução viável,

a) Resolva um programa linear com as variáveis que foram fixadas e arredonde a solução.

Se isso ainda não levar a uma solução viável:

a) Após possivelmente fixar mais variáveis baseado no passo anterior, resolva um programa de inteiro misto por um tempo limitado. Cada um destes passos é possivelmente repetido várias vezes.

Heurísticas Primais usando MIP

Heurísticas Construtívas: Uma heurística Dive-and-Fix

Supomos que temos um problema 0-1. Dada uma solução LP x^*, seja $F = \{j: x_j^* \notin \{0,1\}\}$ o conjunto de variáveis 0-1 que são fracionárias.

Inicialização. Tome a solução LP x^* em algum nó.
Iteração Básica. Enquanto $F \neq \emptyset$,

Faça $i = \underset{\{j \in F\}}{\operatorname{argmin}} \{\min[x_j^*, 1 - x_j^*]\}$ (encontre a variável mais próxima de inteiro).

Se $x_j^* < 0.5$, fixe $x_j^* = 0$ (se próximo a 0, fixe em 0).

Caso contrário, defina $x_j^* = 1$ (se próximo a 1, fixe em 1).

Resolva o LP resultante.

Se o LP for inviável, pare (a heurística falhou). Caso contrário, faça x^* ser a nova solução inicial LP e volte ao início.

Término. Se $F \neq \emptyset$, x^* é uma solução mista inteira viável.

Heurísticas Primais usando MIP

Heurísticas Construtívas: Heurística de Arredondamento por Vizinhança

Dada uma solução ótima da relaxação LP x^*, resolva o PI (Programa Inteiro):

$$\max\{cx : x \in X, \lfloor x_j^* \rfloor \leq x_j \leq \lceil x_j^* \rceil \; for \; j \in N\}$$

Heurísticas Primais usando MIP

Heurísticas Construtívas: Heurística de Feasibility Pump

Inicialização: Seja $x^* = x^{LP}$ uma solução ótima da relaxação linear do problema original (LP).
Iteração: Arredonde para obter um ponto inteiro $\hat{x} = [x_j]$;
Se $\hat{x} \notin P$, projete-o na região viavél P.

Supondo que o IP seja $\max\{cx : Ax \leq b, l \leq x \leq k, x \in Z^n\}$, então o problema projeção pode ser formulado com um PPL da forma $\min\{\sum_j |x_j - \hat{x}_j| : x \in P\}$:

$$\min \sum_{j \in B : \hat{x}_j = \ell_j} (x_j - \ell_j) + \sum_{\in B : \hat{x}_j = k_j} (k_j - x_j) + \sum_{j : \ell_j < \hat{x}_j < k_j} \delta_j$$

$$Ax \leq b$$
$$\delta \geq x - \hat{x}$$
$$\delta \geq \hat{x} - x$$
$$\ell \leq x \leq k$$

onde as variavéis $\delta_j = |x_j - \hat{x}_j|$ para todo j são usadas para modelar as variáveis inteiras situadas entre seus limites.

Heurísticas Primais usando MIP

Heurísticas de Aprimoramento

Dada uma solução viável $\bar{x} \in X$, várias ideias interessantes foram propostas para encontrar soluções melhoradas. Quase todas envolvem a solução de um PIM (Programa Inteiro Misto) no qual muitas variáveis são fixadas ou têm intervalos restritos, de modo que o PIM tenha uma boa chance de encontrar uma solução viável melhor rapidamente.

Heurísticas Primais usando MIP

Heurísticas de Aprimoramento: Local Branching

Dado $\bar{x} \in X$, resolva um PI (Programa Inteiro) no qual as soluções viáveis estão em uma pequena vizinhança de$\bar{x}$. Especificamente, se $X \subseteq \{0,1\}^n$, resolva o PI:

$$\max \left\{ cx : x \in X \cap \left\{ x : \sum_{j:\bar{x}_j = 0} x_j + \sum_{j:\bar{x}_j = 1} (1 - x_j) \leq k \right\} \right\},$$

onde k é um pequeno inteiro.

Inverter os papéis da restrição de ramificação local e a função objetivo leva a uma variante.

Heurísticas Primais usando MIP

Heurísticas de Aprimoramento: Busca de Proximidade

Novamente supomos que $\bar{x} \in X \subset \{0,1\}^n$é dado. Selecionando um parâmetro de melhoria $\delta > 0$, a ideia é encontrar uma solução viável que melhore a melhor solução disponível por pelo menos δ e seja o mais próxima possível da solução inicial $\bar{x}$. Isso leva ao PI:

$$\min \sum_{j:\, \bar{x}_j = 0} x_j + \sum_{j:\, \bar{x}_j = 1} (1 - x_j)$$

$sujeito\ a$

$$\sum_{j=1}^{n} c_j x_j \leq c\bar{x} - \delta$$

$$x \in X.$$

Heurísticas Primais usando MIP

Heurísticas de Aprimoramento: Soft Fixing

Como fixar um número relevante de variáveis sem perder a possibilidade de encontrar boas soluções viáveis?
Para ilustrar melhor esse ponto, suponha que seja dada uma solução heurística 0-1 $\bar{x}$ de um modelo PI 0-1 com n variáveis, e queira-se concentrar em um subproblema central resultante da fixação de pelo menos 90% (digamos) de suas variáveis não nulas em 1. Colocado nestes termos, a questão permite uma resposta simples: basta adicionar ao modelo PI uma restrição linear de fixação suave da forma

$$\sum_{j=1}^{n} \bar{x}_j x_j \geq \lceil 0{,}9 \sum_{j=1}^{n} \bar{x}_j \rceil$$

e aplicar o solucionador "caixa-preta" ao modelo PIM resultante. Dessa forma, evita-se uma fixação demasiado rígida das variáveis em favor de uma condição mais flexível definindo uma vizinhança adequada da solução-alvo atual, a ser explorada pelo próprio solucionador "caixa-preta".

Heurísticas Primais usando MIP

Heurísticas de Aprimoramento: Heurística de Polimento

Motivado pelos algoritmos genéticos, uma lista de no máximo k melhores soluções é armazenada. Duas ou mais soluções $\{x_1, \dots, x_r\}$ são selecionadas e então a ideia é fixar o conjunto $F = \{j \in [1, n]: x_j^1 = x_j^t \; para \; t = 1, \dots, r\}$. Em seguida, novamente o PI

$$\max\{cx : x \in X \cap \{x : x_j = x_j^* \; para \; j \in F\}\}$$

é resolvido. Aqui, a seleção de um par de soluções é obtida escolhendo-se uma primeira solução aleatoriamente e, em seguida, escolhendo-se aleatoriamente uma segunda solução entre as melhores soluções da lista. Mutações são introduzidas fixando aleatoriamente apenas um subconjunto das variáveis em F.

Várias dessas heurísticas foram incorporadas nas versões recentes dos solucionadores de PIM. Existem naturalmente variantes menores e tipicamente o tamanho do conjunto fixado F deve ser ajustado de modo que o PIM resultante resolva rapidamente, mas também produza soluções viáveis.

Créditos

- Integer Programming – Wolsey
- Fischetti, M., Lodi, A. Local branching. *Math. Program., Ser. B* **98**, 23–47 (2003). https://doi-org.ez29.periodicos.capes.gov.br/10.1007/s10107-003-0395-5
- Slides fortemente baseados em cursos dos professores:
  - Celso Ribeiro (DCC/UFF)
  - Laura Bahiense (PESC/COPPE/UFRJ)
  - Pablo Munhoz (UFV – Rio Paranaíba)
  - Pascal Van Hentenryck (Georgia Institute of Technology)